

Spaceadom

Owner's Handbook

Publishing to the Microsoft Store, and the Complete Developer Guide

Version 1.0.107

2026-09-07

Prepared for Nur Ifran Arpon — written so the project can be continued, and the app republished, without any AI assistance.

Table of Contents

PART 1 — PUBLISHING TO THE MICROSOFT STORE	7
1.1 Quick Runbook — How to Publish	7
A. Before you start (5 minutes)	7
B. Partner Center — your part	8
C. Build the Store package	8
D. Submit	9
E. After submission (optional, same day)	10
F. Hand-tests still owed	10
G. If something goes wrong	11
H. State of play (audited 2026-09-07)	11
1.2 Detailed Step-by-Step Reference	14
0. The two-day budget, mapped	14
1. Create the Partner Center developer account (individual, free)	15
2. Reserve the product name	16
3. Find the three identity values	16
4. Paste the three values into <code>identity.json</code>	17
5. Build the package	17
6. Start the submission	18
7. Submit → what happens, and how long it takes	21
8. The WebView2 question	22
What this runbook could not confirm from official docs	23
Sources (all fetched 2026-09-05)	24
1.3 Store Listing Copy — Paste Verbatim	25

Product name	25
Publisher display name	25
Privacy policy URL	25
Support info (Properties page)	25
Short description	26
Description	26
Product features	27
What's new in this version	27
Copyright and trademark info	28
Certification notes — REQUIRED, paste verbatim	28
Age rating questionnaire	30
Category	30
Search terms	31
System requirements	31

PART 2 — THE DEVELOPER GUIDE 33

1. What Spaceadom is, and why it is hard	33
Why this app is harder than it looks	33
2. Your machine: Rust, Node, and the traps	34
Where Rust lives	34
The 0-byte shim trap	35
Node	35
Why <code>npm run build</code> must come before every cargo command	35
3. A map of the repo	36
Documents you will actually read	36
Code	37

Output and archive folders	37
4. Running it	38
The real thing	38
The preview harness	39
5. Build, install, and PROVE	40
The build	40
Installing on the real machine	40
The discipline: never trust an installer exit code	41
How markers work	41
The sandbox differential	42
What a finished ship looks like	42
6. The four build targets	43
<code>setup.exe</code> — the default	43
The <code>.msi</code> — per-machine, and the danger it caused	43
<code>npm run store</code> — the 210 MB offline build	44
<code>npm run msix</code> — a different program	44
<code>npm run portable</code>	45
7. Releasing to GitHub	45
Step 1 — bump three version numbers so they match	45
Step 2 — commit, tag, push	45
What the workflow does	45
The secrets	46
How updates reach users	46
8. Publishing to the Microsoft Store	47
9. Architecture, so you can reason about it	48

The hook thread and its message pump	49
Why the callback may only touch atomics	49
The channel	50
The engine actor	50
The journey of one keypress	50
The two windows	51
The config	52
Boot order in <code>lib.rs</code>	52
How the UI talks to Rust	53
10. The laws, and why each exists	54
The seven keyboard-hook laws	54
Window and overlay rules	55
Design rules	56
A control that does nothing is worse than a missing control	56
The documentation rule	57
Hard rules that are not about code	57
11. How to debug anything	58
Step 0 — read the log FIRST	58
The greps that answer the common questions	58
Step 1 — is that zero real?	59
Step 2 — write a probe with a positive control	60
Step 3 — bisect using <code>all-versions/</code>	60
Step 4 — the honesty rule for intermittent bugs	60
12. How to add a feature without breaking things	61
13. Things you must not lose	61

1. The updater signing key — the worst one	61
2. The Sentry DSN	62
3. The GitHub Actions secrets	62
4. The MSIX identity	62
5. The documentation itself	62
14. DOOMSDAY PROTOCOL — continuing without Claude Code	63
14.1 What is already safe, right now	63
14.2 How to orient yourself, alone	64
14.3 How to use any other AI well	65
14.4 When something breaks: five questions, in order	67
14.5 Rolling back to a known-good build, in two minutes	69
14.6 The five disasters, and how to recover from each	70
14.7 Keeping the documentation habit alive, alone	74
14.8 What to learn next	75
14.9 The short version	76
15. Trap glossary	77
The MSIX container shadow	77
The marker trap	78
The self-matching grep	78
The drained counter	78
Bash mangling a value that begins with a slash	79
The CP1252 curly-quote parse failure	79
WiX validation events that look like installs	80
A read-modify-write one-liner truncating an append-only log	80
An intermittent bug proved fixed by one good hour	81
And one that is not a trap but reads like one: safe mode	81

PART 1 — PUBLISHING TO THE MICROSOFT STORE

Everything needed to finish the Microsoft Store submission is here: the one-page runbook to actually follow, the click-by-click detail behind it, and the exact text to paste into every Partner Center field — so this part of the handbook is enough on its own, with nothing else to open.

1.1 Quick Runbook — How to Publish

One page, in the order you'll actually do things. Longer files behind this one: `RUNBOOK.md`, `SUBMIT-CHECKLIST.md` ("Route B" section), `LISTING.md`. This page alone is enough to finish the job.

Where things stand: the version keeps moving while this is being prepared — **check `package.json`'s "version" field for the exact current number** rather than trusting a figure written here, and use that same number everywhere below (`Spaceadom_<version>_x64.msix`, the git tag, etc). As of this writing it's **1.0.106**, with a code fix for a keyboard-hook edge case (the radial guide could stop appearing for a couple of minutes under a rare condition) already written but not yet built or installed — whatever build you run today will include it, and it'll likely ship as **1.0.107**. Version 1.0.101 already passed a full install-and-run test on this machine (MSIX installed, launched, hook and overlay both worked). You don't need to re-test all of that — see §F for the short list still owed, and the "State of play" section at the end of this file for the full picture.

A. Before you start (5 minutes)

Cleanup from testing on this machine. None of it affects the package you'll build in §C.

1. Remove the local test certificate. This machine trusted a fake certificate to install test copies of the app; it has no purpose now (the Store signs the real package). Open PowerShell **as Administrator**:

```
Get-ChildItem Cert:\LocalMachine\TrustedPeople |  
  Where-Object Thumbprint -eq '710EB524F40B8233E6D485BCCC649839295BA0A1' |  
  Remove-Item
```

2. Optional — the matching private-key copy. The build script also left a private test certificate here (used only to *make* test packages, never to trust them). Harmless to leave; if you want it gone too, no admin needed:

```
Get-ChildItem Cert:\CurrentUser\My |  
  Where-Object FriendlyName -eq 'Spaceadom LOCAL MSIX TEST - not for distribution' |  
  Remove-Item
```

3. Optional — two leftover test files, harmless byproducts the app never reads again:
`%APPDATA%\Spaceadom\packaged-first-run.txt` and `%APPDATA%\Spaceadom\packaged-migration\` (a

folder). Delete or ignore.

4. Ignore — `debug.log` has a few foreign-looking blocks in it, appended by test/probe scripts during this machine's testing. They don't match the normal log format but are harmless; the app never reads its own log back.

B. Partner Center — your part

The one part nobody can do for you: proving who you are and picking the name.

- **Create the account.** Go to <https://storedeveloper.microsoft.com> (this exact URL — the only entry point for the free flow; any other link shows the old \$19 flow). **Get started for free** → **Individual developer (free)** → sign in with a Microsoft account.
- **Identity verification.** Government-issued ID + a selfie, on a phone, in good light, with the real physical document. **Start this first, today** — it's the one step outside your control and Microsoft doesn't publish a turnaround time. Everything below waits on this.
- **Reserve the name.** Once verified: <https://aka.ms/submitwindowsapp> → **New product** → **MSIX or PWA app** → type `Spaceadom` → **Check availability** → **Reserve product name**.
- **Copy three values.** Spaceadom product → **Product management** → **Product identity**: | Partner Center field | Goes into `identity.json` field | | --- | --- | | `Package/Identity/Name` | `name` | | `Package/Identity/Publisher` (starts CN=) | `publisher` | | `Package/Properties/PublisherDisplayName` | `publisherDisplayName` |
- **Paste them in.** From the repo root:

```
copy src-tauri\msix\identity.example.json src-tauri\msix\identity.json
notepad src-tauri\msix\identity.json
```

Fill in exactly those three fields; leave the rest (`_README` is just documentation) alone. This file is gitignored — never committed.

Do not skip this and build anyway — a build without real values prints a large warning and Partner Center rejects it at upload (identity mismatch).

C. Build the Store package

From the repo root, in **PowerShell** (not the Bash tool — a known issue rewrites the signing password there, see `CLAUDE.md`):

```
$env:CARGO_HOME="D:\RUST-DOWNLOADED-HERE\cargo"
$env:RUSTUP_HOME="D:\RUST-DOWNLOADED-HERE\rustup"
$env:PATH="D:\RUST-DOWNLOADED-HERE\cargo\bin;$env:PATH"
```

```
npm run build          # tsc + vite frontend build
npm run tauri build    # the plain release exe
npm run msix           # lays out, packs, and validates the .msix — UNSIGNED
```

Not `npm run store` — that's a different, ~200 MB build for a route this project isn't using. **Do not pass** `-Sign` — that flag is only for testing on a second machine before you have an account; the real submission goes up **unsigned**, because the Store re-signs every package it distributes.

Expected result: `src-tauri\target\release\bundle\msix\Spaceadom_<version>_x64.msix` (<version> is whatever `package.json` currently says — e.g. `1.0.107`), roughly **11 MB** (if it's ~200 MB, `npm run store` ran somewhere upstream by mistake). The build script prints its own verdict — look for exactly:

```
build-msix: VALIDATION PASSED.
```

`build-msix: VALIDATION FAILED` means stop and don't upload.

Do not install this .msix on this machine — it already runs the regular copy, and two copies fight over the spacebar. To test-install, see `RUNBOOK.md §5` / `SUBMIT-CHECKLIST.md`'s second-machine recipe.

D. Submit

Click-by-click detail: `RUNBOOK.md §6–7`. The whole 15-line checklist, in order:

1–8. Account, verification, name reservation, the three identity values, `identity.json`, build, confirm ~11 MB, don't install locally — all done above, §B–C.

- Screenshots already copied into `assets\`: `01-dashboard.png` through `05-launch.png`.
- **Pricing and availability:** Free, all markets, everything else default.
- **Properties:** Category = Productivity, Secondary = Utilities + tools; paste Privacy policy URL, Website, Support URL from `LISTING.md`.
- **Age ratings:** IARC questionnaire, every question **No** except "does your app collect personal information?" → **Yes** (it sends an optional, off-switchable, identifier-redacted crash report — real data collection even with no account or analytics). Exact wording: `LISTING.md`'s Age rating section.
- **Packages:** upload the `.msix`. Expect and ignore a "runFullTrust" notice here — the justification goes in step 15.
- **Store listings:** paste every field from `LISTING.md` (description, short description, features, search terms, copyright). Images:
 - Screenshots: `assets\01-dashboard.png–05-launch.png`
 - Store logo (300×300): `assets\StoreLogo-300x300.png`
 - 1:1 box art (1080×1080), if the slot appears: `assets\AppTile-1080x1080.png`
 - 16:9 hero (optional): `assets\Hero-1920x1080-textfree.png` — the **text-free** one, never the tagline one
- **Submission options:** paste `LISTING.md`'s certification-notes block into **both** "Notes for certification" and "Restricted capabilities" (the latter appears because the app declares `runFullTrust`). Then **Submit for certification**.

Expected certification time: up to 3 business days typically, though Microsoft's FAQ says many clear in a few hours — runs in the background.

The two questions a reviewer is most likely to ask (full answers in `LISTING.md`'s certification notes):

- *"Why `runFullTrust` / a global keyboard hook at all?"* — The feature itself is Space behaving differently in every application, which no sandboxed process can do. No narrower capability grants this.
 - *"It can close other running programs — is that a risk?"* — Only on the user's explicit, twice-confirmed request, only for a fixed built-in list of known keyboard-remapping programs, always a polite close (`WM_CLOSE`) before forcing, never elevates without the standard Windows prompt.
-

E. After submission (optional, same day)

Publishes the current version on GitHub for non-Store users, and is what lets existing older installs self-update to it.

Before tagging: `package.json`, `src-tauri/tauri.conf.json`, and `src-tauri/Cargo.toml` must all already read the **same** version number — CI checks all three against the git tag and refuses to build on a mismatch. Read that number out of `package.json` and use it for both the commit message and the tag below (do not hand-type a number from memory — it's what caused this doc to go stale the last several times).

```
$ver = (Get-Content package.json -Raw | ConvertFrom-Json).version
git add -A
git commit -m "$ver"
git tag "v$ver"
git push
git push --tags
```

The tree has been uncommitted since 1.0.95 (HEAD is `ad23e00`), so this first commit will be unusually large — expected, not a mistake.

What CI then does (`.github/workflows/release.yml`, ~10 minutes): builds both installers, signs the update files, creates a draft release, writes the two updater manifests, uploads everything, then flips the release from draft to public. Watch the repo's Releases page.

F. Hand-tests still owed

Need a human — nothing an AI can click through — on the **installed** app:

- **Win+**. in Notepad while Spaceadom runs — confirm no interference; check `debug.log` for lines starting `emoji panel:.`
 - **Drag a shortcut from one key onto another** — confirm it reassigns, not duplicates.
-

- **"Double" ring layout** on the profile `sexy_tumar_mexy` — confirm the radial guide draws as two rings.
 - **Theme = Auto** — flip Windows light/dark (Settings → Personalisation → Colours) with the dashboard open; both the dashboard and the ring should follow within a couple of seconds, no restart.
 - **First-run tour on a fresh config** — rename/delete `%APPDATA%\Spaceadom\config.json` (back it up first) and confirm the tour card appears.
-

G. If something goes wrong

- **Logs:** `%APPDATA%\Spaceadom\debug.log` — fastest way to see what the app actually did.
 - **Shortcuts die mid-session:** `docs\IF-SHORTCUTS-DIE-AGAIN.md` — this exact symptom happened before; full diagnosis and a one-line check for whether it's the same bug.
 - **App won't start reliably (safe mode):** after three failed launches in a row it disables its own hook and overlay and shows a banner — **"Turn back on"** (re-enables immediately) and **"Report a problem"** (writes a scrubbed zip to `%APPDATA%\Spaceadom\reports\`, never uploads anything).
 - **Need an older version:** every past installer is kept in `all-versions\` — pick one and reinstall it normally (`setup.exe`).
-

H. State of play (audited 2026-09-07)

DONE:

- The MSIX build pipeline (`npm run msix, scripts/build-msix.ps1`) is written, tested against a local placeholder identity, and produces a validated, correctly-sized (~10-11 MB) unsigned package.
 - `identity.example.json`'s three fields match exactly what `build-msix.ps1` reads and what `AppxManifest.xml` needs.
 - All required Store art exists at the right pixel dimensions: `StoreLogo- 300x300.png` (300×300), `AppTile-1080x1080.png` (1080×1080), both hero images (1920×1080), `Hero-2400x1200.png` (2400×1200) — all verified by reading each file's own PNG header, not by filename.
 - `LISTING.md`'s copy has been cross-checked against the current source for every factual claim (special keys, profiles, themes, the update-checker's packaged-mode gate, what data leaves the machine) and corrected where the app had moved on since it was written — see this audit's full report for specifics; the version in your hands may have moved on again since.
 - `PRIVACY.md`'s claims (Sentry redaction, the "Don't send logs" opt-out, the daily update-check interval, no keystroke/binding data in any outbound payload) were checked against the actual Rust source and match.
 - The repo's git remote and `LISTING.md`'s Privacy-policy/Website/Support URLs agree (`github.com/nur-arpon/Spaceadom`), and the Privacy-policy URL currently resolves (verified live, 200 OK) — see the caveat below.
-

- The manifest's `TargetDeviceFamily MinVersion` (10.0.17763.0 / Windows 10 1809) matches what `LISTING.md`'s System requirements section claims.

PENDING (in the order you'd hit them):

- **A fresh build has not been made or installed since the latest code fix landed** (a keyboard-hook edge case where the radial guide could stop appearing for a couple of minutes — fixed in code 2026-09-07, per `PROJECT_STATUS.md`'s newest entry, but not yet built or installed by anyone). Nothing for you to do here except run the normal §C build — the fix is already in the working tree and will be included automatically.
- **All five screenshots in `assets\screenshots\` are stale** — captured 2026-09-05 from v1.0.101. They show the old full-width Settings conflict cards (replaced by a compact grid in 1.0.106) and `04-about.png` has the literal text "v1.0.101" baked into the image. **These need re-capturing from whatever build you actually ship, before you upload them.** Full per-file replacement list: `assets\screenshots\README.md`'s top section.
- **The GitHub repo has been uncommitted since 1.0.95** (current HEAD is `ad23e00`). This means the *live* `PRIVACY.md` a Store reviewer or customer reaches by clicking the Privacy-policy URL today is the stale 1.0.95 version, not the current one this audit checked — the URL resolves (it's not a broken link), but its content is behind. This self-corrects the moment §E's commit-and-push happens; until then, treat the live copy as **not yet representative**.
- `src-tauri\msix\identity.json` **does not exist yet** (only the `.example.json` template does) — this is expected; it's the file that needs your three Partner Center values.
- Everything in §F ("Hand-tests still owed") is, by definition, still owed.

Exact remaining owner actions, in order:

- Register on Partner Center and start identity verification (§B.1–2) — do this literally first, it's the one step outside your control.
- Reserve the product name `Spaceadom` (§B.3).
- Copy the three identity values from Product management → Product identity into `src-tauri\msix\identity.json` (§B.4–5).
- Run the three build commands (§C). This also picks up the pending code fix from PENDING item 1 automatically — nothing extra to do for that.
- Re-capture the five screenshots against this new build and drop them into `assets\screenshots\`, replacing the stale ones (PENDING item 2, full detail in that folder's `README.md`).
- Upload and fill in the submission wizard using `LISTING.md` (§D), then **Submit for certification**.
- Optional, same day: commit, tag, and push (§E) — this is also what fixes PENDING item 3 (the stale live `PRIVACY.md`).

Cross-checked 2026-09-07 against `package.json` (version 1.0.106), `scripts/build-msix.ps1`, `identity.example.json`, `.github/workflows/release.yml`, `PROJECT_STATUS.md`'s newest entry, the actual PNG headers of every file in `assets/` and `assets\screenshots/`, the live git log/git status of this repo, and a fetch of the public Privacy-policy URL. Not independently verified here — see `RUNBOOK.md`'s "What this runbook could not confirm" section: per-field character limits, the exact

Product-declarations checklist, whether the 1080×1080 image slot appears for a Productivity app.

1.2 Detailed Step-by-Step Reference

Written 2026-09-05 against Microsoft's live documentation (every section below names the page it came from, with the fetch date, so a stale claim is easy to spot later). This is Route B — the **MSIX** route described in `SUBMIT-CHECKLIST.md` — because it removes the one blocker Route A has never got past (a paid code-signing certificate): the Store signs the package it distributes, so no certificate purchase is needed at all.

What the owner does, and nothing else: create the account, reserve the name, copy three identity values, paste them into one file, run three commands, then click through a wizard that is 90% already filled in by this document and `LISTING.md`. Everything else in this runbook is either already done (the assets, the copy, the manifest) or is a "click Next" step spelled out below so there's nothing to figure out live.

0. The two-day budget, mapped

When	What	Owner time
Day 1, first thing	§1 Create the developer account	10–20 min active, plus identity-verification wait — see the warning in §1.3
Day 1	§2 Reserve the product name	2 min
Day 1	§3–4 Copy the three identity values, paste into <code>identity.json</code>	5 min
Day 1	§5 Build the package	5 min, one command chain
Day 1	§6 Fill in the submission wizard	30–45 min, mostly pasting from <code>LISTING.md</code>
Day 1 (or as soon as ready)	§7 Submit for certification	1 click
Day 1→3	Certification runs in the background	up to 3 business days — not owner time, but budget for it
Whenever it passes	Publishing	automatic, ~15 minutes after certification passes

The one item that can blow the two-day budget is identity verification in §1.3, because it depends on Microsoft's review, not on anything the owner controls. Start §1 first, before anything else, for exactly that reason.

1. Create the Partner Center developer account (individual, free)

Source: Free developer registration for individual developers

(<https://learn.microsoft.com/en-us/windows/apps/publish/whats-new-individual-developer>) (Microsoft Learn, fetched 2026-09-05, page dated current as of 2026-08-07). **Confirmed: the \$19 individual registration fee is waived** in this flow — the page states it in plain language: "The \$19 registration fee is waived in the new flow."

1.1 The one supported entry point

Go to <https://storedeveloper.microsoft.com> — this is called out as "the only supported entry point for the new \[free\] flow." Reaching account creation any other way (direct Partner Center link, Visual Studio, Xbox) shows the old, paid flow instead.

1.2 Click through

- Click **"Get started for free."**
- Choose **Individual developer (free)** — not "Company account."
- Sign in with an existing Microsoft account, or create a new one.

1.3 Identity verification — the step that can eat the timeline

- **Begin identity verification with a government-issued ID and a selfie.** Do this on a phone, in good lighting, with the original physical document (not a photo of a photo).
- Review the auto-filled profile info pulled from the ID and correct anything wrong.
- Finish account setup and click **"Go to Partner Center dashboard."**

Microsoft's own docs do not state a turnaround time for this verification step, and nothing else fetched during this research pass gave one either — **verify in the wizard / by simply doing it**, and do it literally first, before writing a single Partner Center field, because it is the one part of this whole process outside the owner's control. If it takes more than a few hours, everything downstream in this runbook still works identically once it clears — nothing else in the two-day plan depends on exact timing here, only on it finishing before Day 1 is over.

1.4 After verification

- You'll be prompted for the Microsoft account picker — pick the **same** account used to start the flow.
- Once signed in you land on the **Apps & Games overview** page. If it doesn't appear immediately, Microsoft's doc says to **wait ~5 minutes and refresh**, or go directly to the Partner Center apps and games page (<https://aka.ms/submitwindowsapp>).

That's account creation. No credit card, no yearly fee, nothing to buy.

2. Reserve the product name

Source: Reserve your MSIX app's name

(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/reserve-your-apps-name>)
(fetched 2026-09-05).

- On the Partner Center apps and games page (<https://aka.ms/submitwindowsapp>), click **New product**.
- Click **MSIX or PWA app** (not "Game" — Spaceadom isn't one).
- Type `Spaceadom` and click **Check availability**. A green check mark means it's free.
- Click **Reserve product name**.

That's it — this single click is also what creates the app entry that everything else in this runbook attaches to. Two things worth knowing:

- **The reservation holds for three months** if you don't publish. Not a concern here, but if this stalls past that window, redo this step.
- Reserving the name does **not** by itself set `Package/Identity/Name` (the MSIX identity string) — that's a *related but separate* value you'll copy in §3. Reserving `Spaceadom` as the display name is what makes the Product identity page (next) exist to read from.

3. Find the three identity values

`src-tauri/msix/AppxManifest.xml` needs exactly three account-specific strings before a package can be built, and `identity.example.json`'s own header comment already names where to find them — this section just confirms that's still accurate and adds the click path:

- In Partner Center, open the **Spaceadom** product you just reserved.
- In the left nav, expand **Product management**.
- Click **Product identity**.

That page lists exactly three values, ready to copy:

Partner Center label	Goes into	Looks like
Package/Identity/Name	name	12345NurIfraArpon.Spaceadom
Package/Identity/Publisher	publisher	CN= followed by a GUID
Package/Properties/PublisherDisplayName	publisherDisplayName	Nur Ifra Arpon

This exact page location (**Product management** → **Product identity**) was also independently written into `SUBMIT-CHECKLIST.md` and `identity.example.json`'s own comments earlier the same day this runbook was written — three independent passes agreeing is as close to confirmed as this gets without a live account to click through. If Partner Center's UI has moved this by the time you read it, the values themselves (Identity Name, Identity Publisher, PublisherDisplayName) are what to search Partner Center's own in-product search for.

4. Paste the three values into `identity.json`

```
copy src-tauri\msix\identity.example.json src-tauri\msix\identity.json
notepad src-tauri\msix\identity.json
```

Fill in exactly the three fields the file already has placeholders for:

```
{
  "name": "<Package/Identity/Name from Partner Center>",
  "publisher": "<Package/Identity/Publisher from Partner Center – starts CN=>",
  "publisherDisplayName": "<Package/Properties/PublisherDisplayName from Partner Center>"
}
```

Leave everything else in the file alone — the `_README` block is documentation, not a field, and the version comes from `package.json` automatically. This file is gitignored; it never gets committed and never needs to be, because these three values are meaningless without your Partner Center account attached.

Do not skip this and build anyway. `build-msix.ps1` will run and produce a structurally valid `.msix` using the placeholder `LOCALTEST.Spaceadom / CN=LOCAL-TEST-SPACEADOM-NOT-A-REAL-PUBLISHER` values still sitting in `identity.json` from earlier testing, printing a large yellow warning as it does — and Partner Center will reject that package at ingestion with an identity-mismatch error. The warning exists precisely so this isn't a silent mistake; read it if it appears.

5. Build the package

Three commands, in this order, from the repo root:

```
$env:CARGO_HOME="D:\RUST-DOWNLOADED-HERE\cargo"
$env:RUSTUP_HOME="D:\RUST-DOWNLOADED-HERE\rustup"
$env:PATH="D:\RUST-DOWNLOADED-HERE\cargo\bin;$env:PATH"

npm run build          # tsc + vite frontend build
npm run tauri build    # the plain release exe – NOT npm run store
npm run msix           # lays out, packs, and validates the .msix
```

Not `npm run store`. That target exists only for Route A (it embeds the ~200 MB offline WebView2 installer, which nothing can run from inside a package). `npm run msix` packs the plain release binary from `npm run tauri build` — see "The WebView2 decision" in §8 below for why that's the correct choice and not a shortcut.

The result lands at `src-tauri\target\release\bundle\msix\Spaceadom_<version>_x64.msix`. Confirm it exists and note its size (should be roughly 10 MB, not ~200 MB — if it's the large number, `npm run store` ran instead of `npm run tauri build` somewhere upstream).

This package is deliberately unsigned. Do not try to sign it yourself and do not run `npm run msix -- -Sign` for this submission — that flag exists only for the second-machine structural test described in `SUBMIT-CHECKLIST.md`, using a throwaway local certificate that must never be treated as the real signature. **The Store re-signs every package it distributes with a Microsoft-issued**

certificate at publishing time (confirmed: The app certification process for MSIX app (<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/app-certification-process>), fetched 2026-09-05 — "You don't need to provide your own code signing certificate for Store distribution—the Store handles this automatically"). Uploading the plain unsigned `.msix` is correct and expected.

Do not install this file on this machine. See `CLAUDE.md` and `SUBMIT-CHECKLIST.md` for why — two `WH_KEYBOARD_LL` hooks (this repo's dev install plus a packaged copy) fight over the same spacebar.

6. Start the submission

On the Spaceadom product's overview page, in **Product release**, click **Start submission**. A draft submission appears with six sections, all shown as incomplete. Confirmed order and required/optional status for each, from Create an app submission for your MSIX app (<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/create-app-submission>) (fetched 2026-09-05) — you don't have to do them in this order, but it's the order the page lists them in and the order this runbook follows:

6.1 Pricing and availability

Field	What to set	Why
Markets	All possible markets (the default)	No reason to restrict; a keyboard utility has no region-specific content
Audience	Public audience (default)	
Discoverability	"Make this product available and discoverable" (default)	
Schedule	Release as soon as possible; stop acquisition never (default)	
Base price	Free	
Free trial, sale pricing, org licensing	Leave unset	Not applicable to a free app

Nothing here needs to deviate from the defaults except confirming **Free** as the base price. This page is "Required" but every field has a working default per Microsoft's own submission checklist.

6.2 Properties

Field	What to set
Category	Productivity — see <code>LISTING.md</code> 's Category section for the exact justification against Microsoft's own category definitions
Subcategory	None available under Productivity — leave blank
Secondary category	Utilities + tools

Field	What to set
Privacy policy URL	https://github.com/nur-arpon/Spaceadom/blob/main/PRIVACY.md — required because the app's declared capabilities can access input; if Partner Center doesn't force this, enter it anyway
Website	https://github.com/nur-arpon/Spaceadom
Support contact info	https://github.com/nur-arpon/Spaceadom/issues
Game settings	Not shown — only appears for the Games category
Display mode	Leave everything unchecked — Spaceadom is not a Mixed Reality / HoloLens / 4K-HDR app
Product declarations	Review the checkbox list live in the wizard. If any privacy, data collection, or diagnostics-related declarations exist, enable them and note that crash diagnostic data is collected and sent to Sentry (with personal identifiers redacted), with user opt-out available via the "Don't send logs" setting. Verify in the wizard — this list wasn't itemised in the fetched docs
System requirements	See <code>LISTING.md</code> 's System requirements section — this is a hardware-feature checklist (memory, DirectX, etc.), not an OS-version field; the OS minimum comes from the package manifest automatically (Windows 10 1809+, already set)

6.3 Age ratings

Source: Age ratings for MSIX app

(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/age-ratings>) (fetched 2026-09-05). **This is a real IARC (International Age Ratings Coalition) questionnaire**, not a Microsoft-only checkbox list — confirmed from the doc: "we share your publisher display name and email address with IARC," and you'll separately receive a rating-confirmation email from IARC once published.

- Answer the first question (app category) with whichever option is closest to a utility / productivity tool.
- Answer most follow-up questions **no** (violence, user-generated content, chat/social, advertising, in-app purchases, location, gambling) — but see `LISTING.md`'s Age rating questionnaire section for the one exception: the data collection question, which must be answered **yes** with a full explanation of what diagnostic data the app collects, how it is redacted, and how users can opt out. See the section for the exact wording and the caveat about Partner Center possibly overriding the answer.
- Click **Save and generate**. You'll see all assigned ratings immediately for every market.

This step is required — the app cannot be submitted without it.

6.4 Packages

- Click **Upload** (or drag-and-drop) and select the `.msix` file from §5.

- Wait for it to validate. Partner Center runs the same security, technical-compliance, and structural checks certification will later run, at a lighter level, so problems can surface here first.
- **Expect a "runFullTrust" notice.** Partner Center detects declared restricted capabilities on upload and flags them — this is expected and is not itself a failure. The justification text goes on the **Submission options** page (§6.6), not here.
- **Device family availability:** leave at the default (all supported).

Note from Microsoft's own docs: this section can show **"Incomplete" even after the package itself shows "Validated"** — package validation and section completion are two different checks. Don't assume something is wrong if the section header doesn't turn green the instant the upload finishes; other required sections (Store listings, in particular) also feed into whether Packages reads as complete.

6.5 Store listings

Everything textual here comes from `LISTING.md`, field for field, with the corrected 2026 limits already applied (short description, features, search terms, copyright). Images come from `to-publish-in-microsoft-store/assets/` — see that folder's `README.md` for exactly which file goes in which upload slot. **Real screenshots already sit in `assets/screenshots/` (`01-dashboard.png` through `05-launch.png`) — but as of this writing they were captured from an older build (`v1.0.101`) and must be recaptured before submission: they show the old full-width conflict-card layout (replaced by a side-by-side grid in `1.0.106`) and `04-about.png` has the old version number baked into the visible About row.** See `assets/screenshots/README.md` for the exact replacement list. At least one screenshot is the one image Partner Center actually requires to consider this section complete.

Field	Source
Description	<code>LISTING.md</code> § Description
What's new in this version	Leave blank — first submission
Product features	<code>LISTING.md</code> § Product features
Screenshots (≥1, up to 10, 1366×768+)	<code>assets/screenshots/01-dashboard.png–05-launch.png</code> — re-capture these from the current build first, see the note above
Store logo (1:1 App tile icon, 300×300)	<code>assets/StoreLogo-300x300.png</code>
1:1 box art (1080×1080), if shown	<code>assets/AppTile-1080x1080.png</code> — may not appear; see <code>assets/README.md</code>
16:9 Super hero art (1920×1080), optional	<code>assets/Hero-1920x1080-textfree.png</code> — the text-free one, not the tagline one
Short description	<code>LISTING.md</code> § Short description
Search terms / Keywords	<code>LISTING.md</code> § Search terms (already trimmed to 7)
Copyright and trademark info	<code>LISTING.md</code> § Copyright and trademark info

Do not upload `Hero-1920x1080.png` **or** `Hero-2400x1200.png` (the ones with the tagline baked in) to the Super Hero art slot — Microsoft's own guidance for that field says not to include text, and `assets/README.md` explains why a separate text-free version exists for exactly this reason. Those two files are for the README and social use, not Partner Center.

6.6 Submission options

Source: Manage submission options for MSIX app

(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/manage-submission-options>)
(fetched 2026-09-05).

- **Publishing hold options** — leave at the default ("publish as soon as it passes certification"). No reason to hold this back.
- **Notes for certification** — paste `LISTING.md`'s certification-notes block here (or at minimum the "HOW TO REPRODUCE THE MAIN FEATURE" section of it — a real person reads this, and Microsoft's own guidance says testers appreciate short, clear repro steps).
- **Restricted capabilities** — this section only appears because the package declares `runFullTrust`. Paste `LISTING.md`'s full certification-notes block here. Confirmed from the docs: "For each capability, tell us why your app needs to declare the capability and how it is used... this may add some additional time for your submission to complete the certification process." Budget for that extra time inside the up-to-3-business-day window in §7, not on top of it.
- **Submission notification audience** — the account owner is always notified automatically; add other team members here only if there are any (there aren't, for this account).

Once all six sections show complete, the **Submit for certification** button appears on the application overview page.

7. Submit → what happens, and how long it takes

Source: The app certification process for MSIX app

(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/app-certification-process>)
(fetched 2026-09-05).

Certification typically takes up to three business days, though Microsoft's own FAQ says it's "usually completed within a few hours" for many submissions. Three phases run in sequence:

- **Security tests** — malware/virus scan of the package.
- **Technical compliance** — the Windows App Certification Kit, run by Microsoft against the uploaded package. (`SUBMIT-CHECKLIST.md` already describes running the WACK locally first, on a second machine, as a way to catch the same failures before they cost a submission cycle.)
- **Content compliance** — a human reviewer checks the listing and the app's behaviour against Store Policy, reading the certification notes from §6.6 as they go. This is the phase the restricted-capability justification and the reproduction steps are written for.

After certification finishes you get a **report** — pass or fail, and if fail, which specific test or policy. A failure is not final: fix the specific issue named in the report and create a **new submission**; you do

not have to redo the account or the name reservation.

After it passes, publishing takes a few minutes and the listing becomes visible to customers within about 15 minutes on average. You'll get an email and an Action Center notification either way.

Common failure reasons for a `runFullTrust` + keyboard-hook app, and how this submission's notes address each one

Likely reviewer objection	Where it's already answered
"Why does this need <code>runFullTrust</code> / a global hook at all?"	LISTING.md certification notes §1 — the feature <i>is</i> Space behaving differently everywhere, which no narrower capability grants
"The app also reads keys via a plain page listener sometimes — why?"	LISTING.md certification notes §2 — Windows never delivers the global hook a keystroke typed while the app's own window has focus, so a page-side fallback covers exactly that gap, with a dedupe against the hook and one documented, harmless behavioural difference
"It can close other programs — is that a stability or security risk?"	LISTING.md certification notes §4 — user-request only, twice-confirmed, fixed built-in list, <code>WM_CLOSE</code> before force, never elevates silently, source file named
"Does it log keystrokes?"	LISTING.md certification notes §1 and §PRIVACY POLICY — reads key codes only, keeps no history, no network code
"The app doesn't do anything visible when I open it" (a reviewer expecting an obvious first screen)	LISTING.md's "HOW TO REPRODUCE THE MAIN FEATURE" steps — walk the tester through hold-Space-tap-letter explicitly, since the core interaction (holding Space) is not discoverable by clicking around
"Why is there no visible login / account?"	There isn't one, by design — PRIVACY.md and the age-rating answers both say so consistently; consistency across the submission is what a reviewer is checking for
WACK flags missing scaled asset variants (<code>.scale-200</code> , etc.)	Known and accepted — see <code>SUBMIT-CHECKLIST.md</code> 's "What is deliberately simplified" section; Windows scales the 100%-scale assets that are shipped, so this is a WACK <i>warning</i> , not expected to be a certification <i>failure</i> , but verify in the wizard/WACK report since it hasn't been observed against a real submission yet

8. The WebView2 question

The package ships **no WebView2 runtime**, and this section exists to record that this was a considered decision, not an oversight — cross-checked against `src-tauri/msix/AppxManifest.xml`'s own header comment and against Microsoft's live WebView2 distribution docs (fetched 2026-09-05):

- **Is the Evergreen WebView2 Runtime present on certification machines?** Confirmed from Microsoft's docs: "The Evergreen WebView2 Runtime will be included as part of the Windows 11 operating system," and "the vast majority of Windows 10 devices have the WebView2 Runtime installed already." Since Microsoft's own certification kit (the WACK) and certification

pipeline run on current, updated Windows, the runtime being absent there specifically is unlikely — but nothing in the fetched docs states outright "certification machines always have it," so this specific claim is **inference, not a confirmed fact — verify by watching what the certification report says if this submission ever fails on a missing-runtime symptom.**

- **Is there a Store/package framework dependency mechanism for WebView2?** Confirmed **no**, for this distribution path: `win32dependencies: ExternalDependency` is documented as something you use "if you're using **App Installer** to deploy MSIX applications" — that's sideloaded, not a Store-driven install. Nothing in the fetched docs describes an equivalent mechanism that Store-driven installs honor. This matches, word for word, the reasoning already written into `AppxManifest.xml`'s header comment.
- **The fallback, if this is ever wrong:** the Fixed Version WebView2 runtime, bundled directly in the package (confirmed **+250 MB**, matching the manifest comment's estimate exactly). That is an explicit owner decision to make later if needed, not a default — see `SUBMIT-CHECKLIST.md`, "The WebView2 decision."
- **The listing text**, if this ever needs stating to a customer: "Requires the Microsoft Edge WebView2 Runtime, present on Windows 11 and most Windows 10 devices." This has not been added anywhere in `LISTING.md` because the residual risk is small enough that surfacing it prominently would raise more questions than it answers for the vast majority of customers — but it's here, worded and ready, if a WACK run or a real certification failure ever calls for it.

What this runbook could not confirm from official docs

Flagged inline above, collected here for one scan before submitting:

- The exact wait time for identity verification in §1.3.
- Whether "Keyboard" is one of the checkable hardware items on the Properties → System requirements page (§6.2).
- The exact contents of the "Product declarations" checkbox list (§6.2).
- Whether the 1080×1080 "1:1 box art" upload slot appears at all for a Productivity-category app that isn't a game (§6.5, `assets/README.md`).
- A per-term character limit for Search terms/Keywords, if any (§ Search terms in `LISTING.md` — the 7-term *count* limit is confirmed, a per-term length limit is not).
- A character limit for the "Notes for certification" / "Restricted capabilities" free-text boxes (§6.6) — the pasted text is roughly 4,800 characters as of the 2026-09-07 addition of the page-side-fallback section (§2), growing from the 3,233 characters measured 2026-09-05, with nothing in the fetched docs stating a ceiling. **Verify in the wizard** — if it truncates, `LISTING.md` already says which sections to keep first (1 and 4).
- Whether Windows Store certification machines specifically (as opposed to Windows 11/most Windows 10 generally) have the Evergreen WebView2 Runtime (§8) — reasoned as likely, not confirmed as fact.

None of these block starting or completing the submission — each has a stated default or a safe fallback above — but each is called out so nothing here is mistaken for more certain than it is.

Sources (all fetched 2026-09-05)

- Free developer registration for individual developers
(<https://learn.microsoft.com/en-us/windows/apps/publish/whats-new-individual-developer>)
- Reserve your MSIX app's name
(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/reserve-your-apps-name>)
- Create an app submission for your MSIX app
(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/create-app-submission>)
- Add and edit Store listing info for MSIX app (<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/add-and-edit-store-listing-info>)
- App screenshots, images, and trailers for MSIX app
(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/screenshots-and-images>)
- Age ratings for MSIX app
(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/age-ratings>)
- Manage submission options for MSIX app
(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/manage-submission-options>)
- Enter app properties for MSIX app
(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/enter-app-properties>)
- Categories and subcategories for MSIX app (<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/categories-and-subcategories>)
- System requirements for MSIX app
(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/system-requirements>)
- Support info for MSIX app
(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/support-info>)
- The app certification process for MSIX app
(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/app-certification-process>)
- Distribute your app and the WebView2 Runtime
(<https://learn.microsoft.com/en-us/microsoft-edge/webview2/concepts/distribution>)

1.3 Store Listing Copy — Paste Verbatim

Everything below is written to be pasted as-is into the matching Partner Center field. Where a field has a character limit, the count is noted.

Everything below is written to be pasted as-is. Where a field has a character limit, the count is noted.

Product name

Spaceadom

Publisher display name

Nur Ifran Arpon

Must not equal the product name — that is a named rejection cause.

Privacy policy URL

<https://github.com/nur-arpon/Spaceadom/blob/main/PRIVACY.md>

Partner Center requires this as a real, reachable URL, not a file upload — the policy lives in the public repo so the same document that governs the source governs the Store listing. Paste it into the "Privacy policy URL" field under **Properties** → **Support info** (confirmed location, 2026-09-05).

Support info (Properties page)

Three fields on the same **Support info** section as the privacy policy URL above. Website and support contact are both optional for a non-Xbox app, but cheap to fill in and expected by reviewers:

Website

<https://github.com/nur-arpon/Spaceadom>

Support contact info

<https://github.com/nur-arpon/Spaceadom/issues>

A GitHub Issues URL is an accepted form here — Microsoft's own wording is "the URL of the web page where your customers can go for support ... or an email address," which a repo's Issues page satisfies.

Short description

Field limit, verified against Microsoft's current docs (2026-09-05, Add and edit Store listing info (<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/add-and-edit-store-listing-info#short-description>)): **1,000 characters, but only the first ~270 are shown in most Store views before a customer has to tap to expand it — so 270 is the real target, not 1,000.** (This field used to say "max 500" — that was never the current limit; corrected here.)

Hold the spacebar and tap a key to launch, focus or minimise any app. Space + C for Chrome, Space + S for Spotify — one hand, twenty-six keys, no menus. Space on its own always still types a space.

197 characters — comfortably under the 270-character display cutoff, so nothing of it gets truncated in any Store view.

Description

Field limit: 10,000 characters of plain text (confirmed current, same source as above (<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/add-and-edit-store-listing-info#description>)). The text below is roughly 2,900 characters — nowhere near the ceiling. Do not paste HTML, code, or URLs into this field; Microsoft's guidance says those belong in the dedicated Website/Support/Privacy-policy fields instead, not the description.

Spaceadom turns your spacebar into a modifier key.

Hold Space and tap a letter, and the app you bound to it launches. Tap the same combination again and it minimises. Tap once more and it comes back. Your hands never leave the keyboard and you never go looking through a taskbar again.

Tapping Space on its own always types a space. Always. That rule is the whole design: a modifier you already have, that costs you nothing.

HOLD SPACE TO SEE EVERYTHING

Hold the spacebar for a moment and a ring appears around the SPACE key showing every shortcut you have set up, so you never have to remember them.

SET IT UP BY CLICKING A KEYBOARD

The dashboard is a keyboard. Click a key, pick an app or paste a web address, and it is bound. Drag a shortcut onto a key and that works too.

PROFILES

Three sets of bindings — Founders, Gamers, Professionals — or make your own. Space + Right Alt cycles between them, so the same keys can mean different things while you work and while you play.

MORE THAN LAUNCHING

Space + Esc	Boss key: hide every window and mute the PC in one press
Space + `	Shrink the current window into a corner, or fullscreen it
Space + Tab	Picture-in-picture mode that preserves fullscreen apps
Space + Backspace	Force-quit the app in front, even when it is frozen
Space + ,	Put the cursor where you type on this app or page
Space + up up	Jump to the top of what you are reading

Space + down down Jump to the bottom
Space + scroll Fade the window under your cursor to see what is behind it
Space + Right Alt Cycle between your profiles (see PROFILES above)
Space + . Pause Spaceadom entirely; the same combo turns it back on

Press any of them in the app and a card explains what it does.

IT LOOKS HOW YOU WANT

Four themes: Earthy daylight, Warcry in iron and blood, a Starry night that is a real night sky – constellations you can press for a fact about each, an ocean, and a black galleon under a storm – and Auto, which follows Windows' own light/dark setting and switches within seconds when you change it, no restart needed. Auto is the default on a fresh install. Turn Fun mode off and it all becomes plain and quiet. Hide the keyboard entirely and just watch the sky.

YOUR DATA STAYS ON YOUR COMPUTER

Spaceadom has no account and no analytics – nothing watches how you use it. Everything it stores – your bindings and a local log – sits in your own AppData folder, and the privacy policy says exactly what is in it. The only things that ever leave your machine are an optional crash report if something goes wrong (one click to turn off) and, outside the Store, a daily check for a newer version; neither ever contains what you type or what you have bound.

WHY IT NEEDS TO WATCH THE KEYBOARD

The whole point is that Space behaves differently in every application, so Spaceadom uses a system-wide keyboard hook. It reads key codes to decide "typing or shortcut" in microseconds and keeps no history of what you type.

Product features

Field limits, confirmed: up to **20** features, **200 characters** each, no leading bullets (the Store adds its own). The nine below are the longest at 74 characters — plenty of headroom to add more later without touching these.

Hold Space + tap a key to launch, focus or minimise any app
Space on its own always types a space
A radial guide appears while you hold Space
Set up by clicking a picture of your own keyboard
Three profiles, switchable with Space + Right Alt
Boss key, picture-in-picture, force-quit, smart search, window fading
Four themes, including an animated night sky and an auto light/dark mode
Works everywhere in Windows, not just in one app
No account, no analytics – an optional crash report is the only thing sent

What's new in this version

Leave this field blank. Microsoft's guidance is explicit: "if this is the first time you're submitting your app, leave this field blank." Fill it in starting with the *second* submission (1,500-character limit, confirmed).

Copyright and trademark info

Optional field; not required, but cheap to fill in and removes an easy future dispute:

© 2026 Nur Ifran Arpon. Spaceadom and its logo are trademarks of Nur Ifran Arpon. All rights reserved.

Certification notes — REQUIRED, paste verbatim

This is the field that decides whether a manual review approves or rejects. Do not shorten it.

Where this actually goes, corrected 2026-09-05 after reading the current wizard docs: both on the **Submission options** page — not the Packages page. Partner Center shows two separate boxes there once it detects the package's `runFullTrust` declaration: a general **"Notes for certification"** box, and a **"Restricted capabilities"** box specific to `runFullTrust` that only appears because the capability was detected. Paste this entire block into the **Restricted capabilities** box (it exists to justify exactly this: "why your app needs to declare the capability and how it is used," in Microsoft's own words) — and if the wizard also offers a general notes box, paste it there too, or at minimum add a one-line pointer ("see restricted capability justification above") so a reviewer moving between boxes doesn't miss it. No character limit for this field is stated in Microsoft's current docs; **verify in the wizard** — if it truncates, keep sections 1 and 4 (the keyboard hook and the conflict-closing) first, since those are the two a reviewer is most likely to stop on.

WHAT THIS APP DOES AND WHY IT NEEDS THE PERMISSIONS IT USES

1. GLOBAL KEYBOARD HOOK

Spaceadom installs a system-wide low-level keyboard hook (`WH_KEYBOARD_LL`). This is inherent to the feature: the app makes the spacebar act as a modifier key in every application, which cannot be done from inside a single process. The hook reads virtual key codes only, to decide within microseconds whether a press is ordinary typing or a shortcut. It keeps one timestamp and one key code at a time and no history whatsoever. Nothing typed is recorded, stored or transmitted. The only network use is an opt-out crash reporter (Sentry) that receives anonymised crash reports with file paths, URLs and e-mail addresses redacted before sending; the Store build never checks for updates itself (the Store owns updates). Everything is documented in the privacy policy.

2. A PAGE-SIDE KEYBOARD FALLBACK, WHILE THE APP'S OWN WINDOW HAS FOCUS

Windows does not deliver `WH_KEYBOARD_LL` callbacks to a process while that process's own window is the one in focus. So while Spaceadom's own dashboard window is focused, its web page also runs an ordinary in-page keydown/keyup listener that reimplements the same tap/hold/combo logic and feeds it to the same engine — this is the only place the app reads keyboard input any way other than the global hook, and it only ever sees keys typed while the app's own window is focused, exactly like any other application. A private, timestamp-based dedupe with the hook guarantees only one of the two paths ever acts on a given keypress. One user-visible difference exists: because this path cannot suppress-and-replay a keystroke the way the native hook does, releasing a long Space-hold with no combo inside one of the app's own text fields does not re-type a space (the hook path always does). This is documented in the app's own source comments as a known, deliberate divergence, not an oversight.

3. SendInput

The app synthesises keystrokes in two situations: to replay a suppressed spacebar press when a hold turns out to be ordinary typing, and to send an application's own focus shortcut (for example "/" on YouTube, Ctrl+L for a browser address bar). Every synthetic event is tagged with a private dwExtraInfo cookie so the app never re-processes its own input.

4. CLOSING A CONFLICTING PROGRAM

Only one program on a PC can own the spacebar. Spaceadom detects other keyboard-remapping software (PowerToys, AutoHotkey, spacedesk and similar) and lists it in Settings. On the user's explicit request only – never automatically, and behind two confirmations that state what will happen – it can close one of those programs, and optionally remove that program's start-with-Windows entry from HKCU\...\CurrentVersion\Run and the user's Startup folder.

Constraints enforced in code (src-tauri/src/hook/conflict_close.rs):

- It will only act on a process from a fixed built-in list of known keyboard conflicts. Any other process name is refused and the refusal is logged.
- It asks the program to close normally (WM_CLOSE) before forcing it.
- It never elevates silently. If Windows requires elevation, the standard Windows permission prompt is shown and the user may decline.
- Scheduled Tasks and installed files are never touched, and no program is uninstalled or modified.

5. AUTOSTART

Optional, user-controlled from Settings. In this Store package it is the manifest's windows.startupTask (visible and controllable in Settings > Apps > Startup); the app never writes a Run value or a scheduled task when packaged.

6. NO ELEVATION

The app itself never runs elevated and shows no UAC prompt at any point during normal use.

7. PROCESS LAUNCHES (why the package references cmd.exe / powershell.exe)

Spaceadom is an app launcher: it starts the programs, files and links the user bound to keys (CreateProcess / ShellExecute). It also runs PowerShell once in the background to list the Start-menu shortcuts for the app picker, and cmd.exe only to open a folder or restart itself after an update. It never runs scripts from the network, never modifies the registry outside its own HKCU keys, and never touches other software except the single user-confirmed "close a conflicting keyboard program" action described in 4.

HOW TO REPRODUCE THE MAIN FEATURE

1. Install and let the dashboard open.
2. Click any letter key on the on-screen keyboard, choose an app, close the editor.
3. Hold the spacebar for about half a second – a ring of your shortcuts appears.
4. While still holding Space, tap that letter. The app launches.
5. Repeat the combination to minimise it, and again to restore it.
6. Tap Space on its own in any text field to confirm it still types a space.

PRIVACY POLICY

Included as PRIVACY.md and hosted at

<https://github.com/nur-arpon/Spaceadom/blob/main/PRIVACY.md>, the same URL

entered in the listing's Privacy policy URL field. It documents every file written, exactly what the local log contains, what the crash reporter and the update checker each send (the Store build never runs the update checker – Store handles updates itself), and how to delete everything.

Age rating questionnaire

This is a live IARC (International Age Ratings Coalition) questionnaire embedded in Partner Center, not a Microsoft-only form — confirmed from Microsoft's current age-ratings doc (2026-09-05). It shares your publisher display name and email with IARC, and the first question asks which category best describes the app; answer with whichever option is closest to "utility / productivity tool," then answer every follow-up **no**: no violence, no user-generated content, no chat or social features, no data collection, no advertising, no in-app purchases, no location, no gambling. That combination lands at the lowest available rating (historically "3+" / PEGI 3-equivalent) for a content-free utility. Once submitted, the same rating carries forward to every future update automatically — you only retake it if a later version's content changes what these answers say.

The one question that catches people out:

- **"Does your app collect personal information?"** → **Yes**. The app collects diagnostic crash data: error messages, stack traces, file paths, and line numbers from Spaceadom's own source — all with personal identifiers (Windows usernames, system paths, URLs unrelated to the app) redacted before sending to Sentry. No personal data is ever intentionally recorded or shared with third parties beyond the crash-reporting processor, and the user can opt out entirely with the "Don't send logs" setting (off by default, which means reporting is on by default). The app has no accounts, no analytics, no advertising, and no third-party sharing except for crash data to Sentry. The local log records which shortcut was pressed and which app was launched, on the user's own disk; that is documented in the privacy policy. See the Privacy policy URL for the complete list of what the app collects locally and what may leave the device. Note, though, that Partner Center may **override this answer automatically** based on the capabilities detected in the package — if that happens, the privacy policy URL on the Properties page already documents the exact data flow.
-

Category

Primary: Productivity (no subcategory exists under Productivity — Microsoft's category table lists it with subcategory "(None)"). **Secondary: Utilities + tools** (the exact name in the Store's 2026 category list is "Utilities + tools", not "Utilities & tools" — corrected here).

Why Productivity, not Utilities + tools, as primary: Microsoft's own one-line definitions (Categories and subcategories

(<https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/msix/categories-and-subcategories>), fetched 2026-09-05) describe Productivity as "apps which help user to complete a particular task more efficiently" — which is the entire pitch — versus Utilities + tools as "apps which assist user in solving problems or completing specific tasks," illustrated with file managers, calculators and

barcode scanners. Spaceadom is closer to the first description than the second, so it leads there and takes Utilities + tools as the secondary category instead of the reverse.

Search terms

Field limit, confirmed: 7 unique terms maximum. The list this file carried before had nine — over the limit, and would have been silently truncated or rejected. Trimmed to seven, dropping the two most redundant with terms already on the list ("keyboard shortcut" overlaps "keyboard modifier" and "hotkey"; "remap" overlaps the same two):

spacebar, app launcher, window switcher, hotkey, productivity, keyboard modifier, alt-tab replacement

A per-term character limit is not stated anywhere in Microsoft's current docs found during this pass — **verify in the wizard** if it complains about any single term's length; none of the seven above are longer than 19 characters, which should be safe under any historical limit.

System requirements

Correction, 2026-09-05: the minimum OS version is not something you type into a Properties-page text field for an MSIX submission. It comes straight from the uploaded package's own manifest — `src-tauri/msix/AppxManifest.xml's <TargetDeviceFamily MinVersion="10.0.17763.0" .../>`, which is **Windows 10 version 1809**, not 2004 (build 19041) as an earlier draft of this document assumed. Partner Center reads that from the package; there is nothing to reconcile by hand as long as the manifest and this text agree, which they now do.

What the **Properties** → **System requirements** page actually asks for instead is a table of optional hardware declarations (memory, DirectX, graphics, and similar) with **Minimum/Recommended** checkboxes per item — confirmed from Microsoft's current docs, though the exact list of hardware items (whether "Keyboard" is one of them) wasn't visible in the fetched page text, so **verify in the wizard**. If a "Keyboard" item exists, check it under **Minimum hardware** — the entire feature is unusable without a physical keyboard, and there is no touch or mouse-only path. Leaving the whole section blank is also fine (it's optional); it just means the Store won't show a hardware warning to a customer on an unsuitable device.

The Store listing's own **"Additional system requirements"** supplemental field (Properties page → Additional system requirements, under Minimum hardware) is structured as **up to 11 short bullet items, 200 characters each** — confirmed — not one free-text paragraph, so here it is split into items that actually fit that shape:

Windows 10 version 1809 or later, or Windows 11
64-bit (x64) only – not ARM
Roughly 30 MB of disk space
No internet connection required for any feature to work
Only reaches the network for an optional crash report (off in one click) and, outside the Store build, a daily update check

If the wizard instead offers one free-text box (older UI, or a different field than the one described above), the single paragraph below reads fine there:

Windows 10 version 1809 or later, or Windows 11. 64-bit x64 only – not ARM.

Roughly 30 MB of disk space. No internet connection is required for any feature to work – Spaceadom only reaches the network to send an optional crash report (off in one click, see [PRIVACY.md](#)) and, outside the Store build, to check for updates once a day.

PART 2 — THE DEVELOPER GUIDE

Written 2026-09-07, for Nur Ifran Arpon, to be read alone.

This file assumes you are sitting at your own machine with no AI helping you, and that you want to keep building Spaceadom anyway. Everything here is written so you can do that.

It is long. Read it once end to end — about an hour — and after that use it the way you would use a manual: jump to the section you need. Section 14 is the one to re-read on a bad day.

How this file relates to the others. `CLAUDE.md` is the law. This file is the teaching. Where they disagree, `CLAUDE.md` wins and this file is stale — tell whoever is reading it. Nothing here replaces `V14_FIXES_AND_CODE.md`, which is where the actual fixes live.

1. What Spaceadom is, and why it is hard

Spaceadom turns your Spacebar into a modifier key for the whole of Windows.

Hold Space and tap `s`, and Spotify opens. If Spotify is already open behind something else, it comes to the front. If it is already in front, it minimises. That is the Smart Cascade, and it must loop forever without getting confused. Tap Space on its own and you get a space character, always, with no delay you can feel. Hold Space for a moment and a ring of your shortcuts appears on screen; let go and it disappears.

There are three profiles, a dashboard where you click a key on a picture of a keyboard and choose what it launches, a boss key on `Space+Esc` that minimises everything and mutes the sound, a bypass on `Space+` that turns the whole thing off for gaming, and a handful of extra shortcuts. Four themes, one of them an animated night sky. The whole thing runs in the tray and starts with Windows.

The full, binding list of what it must do is `CORE_AIM.md`. **You are not allowed to delete a feature from that list to make something build.** If it is broken, fix it properly.

Why this app is harder than it looks

Three things make Spaceadom difficult, and almost every bug in its history traces to one of them.

One: it is a system-wide keyboard hook. Windows lets a program watch every keystroke on the machine through something called `WH_KEYBOARD_LL`. That callback runs on the OS's input path — *every key on the machine waits for your code to return*. If your callback is slow, typing stutters everywhere. If it is too slow even once, Windows quietly stops calling it and never tells you. The handle stays valid. There is no error. From inside the process, a hook that has been thrown out and a hook nobody has typed into look identical. That single fact is behind PROBLEMs 230, 236, 257, 260 and 262 — five separate multi-day investigations.

And because the hook swallows Space to decide what it means, the app cannot see its own Space either. The dashboard's web page receives `keydown` events the hook never gets, which is why there are now **two** witnesses for a Space hold and a whole set of rules keeping them apart (PROBLEM 259).

Two: it draws a click-through overlay. The ring and the toasts live in a second, transparent, always-on-top window that the mouse passes straight through. Transparent windows on Windows are composited by the OS, not by your page — so when one goes wrong, *the page cannot tell*. Everything inside it reports perfect health while nothing is on screen. A 560×320 element with `blur(34px)` on it made the entire overlay window compose zero pixels (PROBLEM 37). A `hide()` call with no logging hid the window a second before the toast arrived, and three builds of animation work played out invisibly (PROBLEM 135). This is why the overlay logs its own size, position and visibility on every move, and why you must never delete that logging.

Three: the installer has to replace an app that is currently running. You are always updating Spaceadom while Spaceadom is holding its own files open. Windows Installer's answer to that is to defer the file swap to the next reboot — so the installer exits 0, reports success, and nothing changed. That happened four separate times (PROBLEM 127) and is the origin of the single most important habit in this project: **never trust an installer's exit code**.

Everything in section 5 exists because of that sentence.

2. Your machine: Rust, Node, and the traps

Where Rust lives

Rust is **not** in the usual place. It lives at `D:\RUST-DOWNLOADED-HERE`, and it must stay there — the registry cache alone is about 1.9 GB and your C: drive does not have room to spare.

```
D:\RUST-DOWNLOADED-HERE\  
  cargo\      <- CARGO_HOME   (bin\ plus the registry cache)  
  rustup\     <- RUSTUP_HOME  (toolchains\)
```

There are user environment variables set for this already, but any shell that does not have them will happily install a *second* Rust into `C:\Users\beamu\.cargo`. So set them explicitly at the top of any build session.

PowerShell — this is the form you should use:

```
$env:CARGO_HOME="D:\RUST-DOWNLOADED-HERE\cargo"  
$env:RUSTUP_HOME="D:\RUST-DOWNLOADED-HERE\rustup"  
$env:PATH="D:\RUST-DOWNLOADED-HERE\cargo\bin;$env:PATH"  
cargo --version      # expect cargo 1.97.1
```

Bash (Git Bash / MSYS), if you ever need it:

```
export CARGO_HOME="D:/RUST-DOWNLOADED-HERE/cargo"  
export RUSTUP_HOME="D:/RUST-DOWNLOADED-HERE/rustup"  
export PATH="/d/RUST-DOWNLOADED-HERE/cargo/bin:$PATH"
```

```
cargo --version
```

Use PowerShell for builds. Bash works for reading and grepping, but it rewrites any value that starts with a `/` into a Windows path before handing it to a native `.exe`. Your signing password is base64, so roughly one password in three starts with `/`. That cost a full day on 2026-09-05: the password was always correct, and every attempt reported `Wrong password for that key`, because MSYS had turned it into `C:/Program Files/Git/...` on the way through. See section 15. **Any secret handed to a Windows program goes through PowerShell.**

The 0-byte shim trap

This machine does this reproducibly, and it looks exactly like Rust was never installed.

Symptom: `cargo`, `rustc` and `rustup` all fail with *"The system cannot find the file specified."*

Cause: the shim files in `cargo\bin` are **0 bytes**. Windows reports that exact error when you try to execute an empty file. `Test-Path` returns `True` for them, so every "is cargo installed?" check says yes and sends you the wrong way.

Check the length, not the existence:

```
Get-ChildItem D:\RUST-DOWNLOADED-HERE\cargo\bin | Select-Object Length,Name
```

Repair. `rustup default stable` does *not* fix it — `rustup` sees files present and skips them. A shim is just a copy of `rustup.exe` that decides what to do from its own filename, so you rebuild them by copying:

```
$bin="D:\RUST-DOWNLOADED-HERE\cargo\bin"; $src="$bin\rustup.exe"
$shims=@('cargo','rustc','rustdoc','rustfmt','cargo-fmt','cargo-clippy',
        'clippy-driver','cargo-miri','rls','rust-analyzer','rust-gdb',
        'rust-gdbgui','rust-lldb')
foreach($s in $shims){ $t="$bin\$s.exe"
    Remove-Item $t -Force -ErrorAction SilentlyContinue; Copy-Item $src $t -Force }
cargo --version
```

Node

Node 20 or newer, with `npm`. `npm ci` (or `npm install`) once after a fresh clone. There are only four dependencies in total — `@tauri-apps/api`, `@tauri-apps/plugin-opener`, the Tauri CLI, Vite and TypeScript. That is deliberate.

No React. No Tailwind. Ever. The frontend is vanilla TypeScript. This is a standing decision, not something nobody got round to.

Why `npm run build` must come before every cargo command

```
npm run build          # tsc, then vite build --outDir dist2
```

At Rust compile time, the macro `tauri::generate_context!` reads the folder named by `frontendDist` in `tauri.conf.json`, which is `../dist2`. It embeds those files into the binary. If `dist2` is missing or stale, you either get a binary containing yesterday's UI, or you get this:

error: proc macro panicked

...with no cause named at all. That message means "run `npm run build` first" almost every time.

Two more folders exist and are **not** what ships: `dist/` and `dist-stale/`. They are leftovers on disk. `dist2` is the real one.

3. A map of the repo

Root: `D:\Claude-Projects\SpaceToggle-V14`. (The folder is still named after the old version. The app is Spaceadom.)

Documents you will actually read

File	What it is
CLAUDE.md	The law. Architecture, every hard rule, the build steps, the testing laws. Long because most of it was learned by shipping a bug.
CORE_AIM.md	The feature contract. Nothing on this list may be removed to make something build.
V14_FIXES_AND_CODE.md	Every problem ever solved: symptom, root cause, exact file, the actual code, how it was verified. ~1.7 MB. Search it before diagnosing anything.
PROJECT_STATUS.md	The dated dev log, newest at top, append-only. Never delete an entry.
NATIVE_SAFETY.md	The do-not-touch table for Win32 calls. Exists because this app once broke your touchpad gestures.
docs/IF-SHORTCUTS-DIE-AGAIN.md	Paste this whole file to any AI before it investigates the shortcuts dying. Three wrong hypotheses are already disproved in it, with numbers.
RELEASE_READINESS.md	What is done, what is left, and the Store blockers. Written 2026-08-26 — treat its dates seriously.
PRIVACY.md	What leaves the machine. Linked from the Store listing.
CONTRIBUTING.md, SECURITY.md, LICENSE	Repo hygiene. The licence is source-visible, not open source.
THIRD-PARTY-NOTICES.md	Generated from 571 real dependencies.
README.md	The public front page.
AI_HANDOFF.md, HANDOVER_PROMPT.md, FINAL_RELEASE_README.md, WHAT_HAPPENED.md, V13_TO_V14_METHOD.md, OVERLAY_ACHIEVED.md, OVERLAY_RUST_HTML_CHANGES.md, SHIPPING_AUDIT.md, FEATURES_NOW_POSSIBLE.md, FUTURE_IDEAS.md	Fossils , marked SUPERSEDED at the top. Read them for the reasoning they record, never for current fact.

Code

Path	What it is
src/	The dashboard UI. Vanilla TypeScript. <code>main.ts</code> wires everything; <code>components/</code> holds the keyboard matrix, key editor, settings panel, profile editor, tour, toasts, night sky.
src/overlay.ts + overlay.html	The transparent on-demand HUD and toast surface. A separate window from the dashboard.
src/preview.ts + preview.html	Dev-only visual harness (section 4). Never shipped.
src/styles/	<code>design-system.css</code> (the tokens), <code>themes.css</code> , <code>characters.css</code> , <code>starry-sky.css</code> , <code>overlay-earthy.css</code> .
src-tauri/src/	The Rust backend. See section 9.
src-tauri/tauri.conf.json	App identity, the two window definitions, bundle targets, CSP, the updater's public key and endpoint.
src-tauri/tauri.store.conf.json	The override that makes the ~210 MB Store build.
src-tauri/Cargo.toml	Rust dependencies. Also holds one of the three version numbers.
src-tauri/capabilities/default.json	Which windows are allowed to use which Tauri APIs. A window missing from here is deaf — every <code>listen()</code> in it fails silently.
src-tauri/wix/main.wxs	The custom WiX template for the <code>.msi</code> . Forked from the stock file with exactly four changes, each marked <code>SPACEADOM CHANGE n</code> .
src-tauri/installer-hooks.nsh	NSIS hooks — kills the running app before install, asks "keep your settings?" on uninstall.
src-tauri/msix/	<code>AppxManifest.xml</code> and <code>identity.example.json</code> for the Store package. <code>identity.json</code> is gitignored and private.
src-tauri/symbols/	Where the <code>.pdb</code> is staged so crash reports resolve to real line numbers.
src-tauri/build.rs	Writes a stub <code>.pdb</code> if one is missing, so <code>cargo test</code> does not fail.
src-tauri/.tauri/	The updater signing key. Gitignored. See section 13.
scripts/	Build, install, proof and probe scripts. See sections 5–7.
.github/workflows/release.yml	The release pipeline.
index.html, vite.config.ts, tsconfig.json, package.json	Frontend build config. <code>package.json</code> holds another of the three version numbers.

Output and archive folders

Path	What it is
dist2/	The built frontend. This is what gets embedded in the exe.

Path	What it is
dist/, dist-stale/	Leftovers. Not used.
src-tauri/target/	Rust build output, ~4.6 GB. Never committed. Installers appear under <code>target/release/bundle/</code> .
all-versions/	Every installer ever built , 200+ of them. Your rollback shelf — see section 14.
share-spaceadom/	The current installers plus a read-me and the privacy policy, for handing to a friend. Refreshed automatically after every build.
to-publish-in-microsoft-store/	Store paperwork: <code>START-HERE.md</code> , <code>RUNBOOK.md</code> , <code>LISTING.md</code> , <code>SUBMIT-CHECKLIST.md</code> , and the screenshots and art.
winget/	Prepared winget manifests. Never submitted.
skill-package/	The AI skill that governs work here; copy it to <code>.claude/skills/</code> if it goes missing.
_probe/, _recovered/, _config-rescue/, and the _*.txt/_*.json/_*.log files at the root	Evidence from past sessions — probe output, config snapshots taken before installs, build logs. Clutter, but <i>evidence</i> . Do not tidy them away.
install-check.txt, install-proof.txt, preinstall-probe.txt, postinstall-probe.txt, config-check.txt	Where the install and probe scripts write their findings. Overwritten each run.
install-v11.ps1	The original AutoHotkey version. The functional gold standard — when behaviour is ambiguous, match v11.

4. Running it

The real thing

```
$env:CARGO_HOME="D:\RUST-DOWNLOADED-HERE\cargo"
$env:RUSTUP_HOME="D:\RUST-DOWNLOADED-HERE\rustup"
$env:PATH="D:\RUST-DOWNLOADED-HERE\cargo\bin;$env:PATH"
cd D:\Claude-Projects\SpaceToggle-V14
npm run tauri dev
```

This starts Vite on `http://localhost:1420`, compiles the Rust in debug mode and launches the app. It installs a real keyboard hook, so Space really does become a modifier while it runs.

Kill the installed copy first, or you have two hooks fighting for the spacebar:

```
taskkill /IM spaceadom.exe /F
```

Two things behave differently in a debug build, both on purpose:

- `st-updater-endpoint.txt` beside the exe (first line: a manifest URL) makes the updater point somewhere else and accept a self-signed certificate. This only works in a **debug** build — a release build ignores the file entirely and says nothing about it, because that file sits in a

directory anything running as you can write to.

- Debug builds are slower. The keyboard hook has a deadline measured by Windows. Do not conclude anything about hook timing from `tauri dev`.

The preview harness

```
npm run dev
```

Then open `http://localhost:1420/preview.html` in a normal browser.

This renders the **real** dashboard components against a stub config, with no Rust behind them. It exists so you can look at and adjust the UI without a four-minute Rust build in the loop. It is not a Vite build input, so it can never ship.

Query flags — stack them with `&`:

Flag	What it does
<code>?dark</code>	Start in the Nocturne / dark palette
<code>?theme=<name></code>	Pick a specific theme
<code>?osdark</code>	Pretend Windows is set to dark, for testing theme "Auto"
<code>?gear</code>	Open the settings panel
<code>?expand</code>	Open the settings panel expanded (wide)
<code>?profiles</code>	Open the profile popover
<code>?specials</code>	Open the special-shortcuts tray
<code>?editor / ?editor=<key></code>	Open the key editor
<code>?sky</code>	Show the night scene
<code>?conflict</code>	Show the "another keyboard app is running" cards
<code>?fun</code>	Fun mode — the character animations on toggles and sliders
<code>?tour</code>	Run the first-run walkthrough
<code>?about-open</code>	Open the About section
<code>?rollback</code>	Show the About screen's rollback button
<code>?portable</code>	Pretend this is a portable build (greyed-out startup row)
<code>?double / ?wide</code>	Ring layout variants
<code>?ownwindow</code>	Exercise the own-window key fallback

There are also console helpers: `window.__previewEmit("st-launched", {...})` fires a backend event by hand, `window.__previewStartupProbe()` and `window.__previewThemeProbe()` walk every state the backend can return.

What the harness can prove: layout, spacing, colours, contrast, focus rings, keyboard navigation, that a component renders every state, that a CSS change did what you meant.

What the harness cannot prove — and this matters:

- **Anything about the overlay.** The overlay's failure mode lives in the Windows compositor, not in the page. A browser will happily render an overlay layout that composes zero pixels on the real machine. Hold Space and *look* before shipping any visual change to the ring or a toast.
 - **Anything about the keyboard hook.** There is no hook here.
 - **Anything about real Rust behaviour.** The stub config answers every `invoke()` with a canned value.
 - **Themes toggled after load.** Flipping `body.classList.add("nocturne")` on a page that is already loaded leaves some `color-mix()` backgrounds reading the old palette. Load the harness already in the theme you want (`?gear&expand&dark`) instead. That was measured on 2026-09-07 and was nearly filed as a product bug.
-

5. Build, install, and PROVE

This is the most important discipline in the project. Read it twice.

The build

```
$env:CARGO_HOME="D:\RUST-DOWNLOADED-HERE\cargo"
$env:RUSTUP_HOME="D:\RUST-DOWNLOADED-HERE\rustup"
$env:PATH="D:\RUST-DOWNLOADED-HERE\cargo\bin;$env:PATH"
cd D:\Claude-Projects\SpaceToggle-V14
npm run build
npm run tauri build
```

Output lands in:

```
src-tauri\target\release\bundle\nsis\Spaceadom_<version>_x64-setup.exe
src-tauri\target\release\bundle\msi\Spaceadom_<version>_x64_en-US.msi
```

After the bundle step, `scripts/archive-build.mjs` runs automatically and copies both into `all-versions/` and refreshes `share-spaceadom/`. That is wired into the build precisely because it used to be a manual step and was forgotten for five versions in a row.

To also produce the `.sig` files the updater needs, set the signing env vars first — **from PowerShell, never Bash:**

```
$env:TAURI_SIGNING_PRIVATE_KEY = (Get-Content "src-tauri\.tauri\spaceadom.key" -Raw)
$env:TAURI_SIGNING_PRIVATE_KEY_PASSWORD = (Get-Content "src-tauri\.tauri\spaceadom.key.password.txt" -Raw).Trim()
npm run tauri build
```

Without those two variables the build succeeds and simply writes no `.sig`. That is not an error you will see; you have to check the files exist.

Installing on the real machine

```
Start-Process explorer.exe -ArgumentList 'D:\Claude-Projects\SpaceToggle-V14\scripts\install-real.cmd'
```

Then read `install-check.txt` in the repo root.

Why through `explorer.exe`? Because an agent shell (and some other containerised shells) run inside an MSIX container that redirects `%LOCALAPPDATA%` and virtualises `HKCU`. An install run from inside it lands in the container, and every check made *from the same shell* then agrees with itself and is wrong. `explorer.exe` runs outside the container. From your own normal PowerShell window you can run the `.cmd` directly — but going through `explorer.exe` always works, so make it the habit.

Before each release, edit one line in `scripts/install-real.cmd`: the `set SETUP=` line names the exact installer filename, version and all. It is hand-edited, not derived.

The discipline: never trust an installer exit code

`setup.exe /S` has exited 0 having replaced nothing, four separate times. The app was running during its own upgrade and the file swap got deferred. The NSIS PREINSTALL hook now kills the app first, and `install-real.cmd` kills it again for good measure — but the rule stands for any installer, forever:

Verify the installed exe by version stamp AND by an ASCII content marker before believing a fix shipped.

A verdict taken on an unverified build is worse than no verdict, because it discredits a fix that actually worked.

How markers work

Rust `log::` format strings are plain ASCII inside the compiled binary. You can search for them without running anything:

```
$exe = "$env:LOCALAPPDATA\Spaceadom\spaceadom.exe"
$txt = [Text.Encoding]::ASCII.GetString([IO.File]::ReadAllBytes($exe))
$txt -match 'MECHANISM \(\PROBLEM 260\): when a WH_KEYBOARD_LL callback overruns'
```

So the proof of "did my fix ship?" is: pick a long, distinctive sentence from the code you changed, and check the installed exe contains it.

Four rules, each learned the hard way:

1. Use a long `log::` format string, never a short identifier. A short literal that only ever gets copied into a `String` may never exist contiguously in the binary at all. `st-hud-pointer` — 14 bytes, a thread name — tested **False in a freshly built exe that certainly contained it** (measured 2026-08-27): the compiler builds it at runtime with two overlapping `mov` instructions, so the bytes never sit together on disk. Longer names in the same file survived fine. Pick a whole sentence.

2. Confirm the marker is present in the FRESH build BEFORE its absence anywhere else means anything. This is the whole game. If you skip it, a `False` reads as "the fix did not ship" when it actually means "that marker was never findable" — and that is how a working build gets thrown away. The proof table every ship writes looks like this:

	installed old version	freshly built exe	installed new version
new marker	False	True	True

	installed old version	freshly built exe	installed new version
~20 old control markers	True	True	True

The old controls are there so that a wall of `False` tells you the *scan* is broken rather than the build.

3. Frontend markers do not work this way at all. Tauri v2 compresses the embedded `dist2` assets, so CSS class names and JS strings are not findable inside the exe. `st-hud-glow` tests `False` in a binary that certainly contains it (measured 2026-08-20). A frontend string that *is* found is one that also exists in the Rust source. Use this chain instead:

- `grep` the marker in `dist2\assets\*` — proves it is in the bundle;
- confirm the installed exe's `LastWriteTime` is **later** than the newest file in `dist2` — proves the exe embedded *that* bundle;
- confirm the installed exe's version stamp.

`scripts/install-proof.ps1` does all of this, plus the registry read-back, plus the law-6 log assertion, in one pass. `install-real.cmd` calls it.

4. Read it all from outside the sandbox. See below.

The sandbox differential

A containerised shell redirects `%LOCALAPPDATA%` at the *filesystem* layer. The path string is byte-identical inside and outside — so printing `%LOCALAPPDATA%` proves nothing. An agent did exactly that on 2026-08-27, concluded it had escaped the container, and was right only by luck.

The only valid proof is a differential: read the *same path string* from both contexts and compare what comes back. The real measurement that night:

```
%LOCALAPPDATA%\Spaceadom\spaceadom.exe
  in-sandbox      v1.0.53, 14,109,184 bytes
  via explorer.exe v1.0.86, 18,868,224 bytes
```

Two different files at one path. That is the only thing that demonstrates redirection.

Generalise it, and this is the sentence to carry into everything else in this guide: **a check that cannot produce a negative result is not a check.**

What a finished ship looks like

- Gates green: `cargo test --lib, cargo clippy --all-targets, npx tsc --noEmit, npm run build`.
- Version bumped in all three files (section 7).
- Preinstall probe run — snapshot the *old* installed exe, so the new markers can be proven genuinely new.
- Build, signed, from PowerShell.
- New markers confirmed **True in the fresh exe** and **False in the old installed one**, plus the ~20 controls still `True`.
- Install through `explorer.exe`; read `install-check.txt`.

- Postinstall probe — confirm the machine is *running* the new build, not just holding the file.
- **Hold Space with the dashboard focused and look for the ring.** If you cannot do this, the report says **UNPROVEN**, in capitals. See law 6 in section 10.

A fix that is not installed does not exist. You boot from `%LOCALAPPDATA%\Spaceadom\`, never from `target\release\`. Testing from the repo build is fine; the session is not finished until the installer has run and the installed exe has been verified. Five hours of fixes once failed to reach your startup exactly this way (PROBLEM 42).

6. The four build targets

Command	Produces	For
<code>npm run tauri build</code>	<code>setup.exe</code> (NSIS) and <code>.msi</code> (WiX)	Normal releases. The <code>setup.exe</code> is the one you hand out.
<code>npm run store</code>	<code>...-setup-STORE.exe</code> , ~210 MB	The Microsoft Store's Route A (unpackaged EXE listing). Blocked on code signing.
<code>npm run msix</code>	<code>Spaceadom_<v>_x64.msix</code> , ~10 MB	The Microsoft Store's Route B — the one you are actually taking.
<code>npm run portable</code>	<code>...-portable.zip</code>	Someone who wants no installer at all.

`setup.exe` — the default

NSIS, per-user, installs into `%LOCALAPPDATA%\Spaceadom`, `nsis.installMode = "currentUser"`, **no UAC prompt ever**. About 5.6 MB. This is the recommended installer and the one the daily updater uses.

The `.msi` — per-machine, and the danger it caused

WiX, per-machine, installs into `C:\Program Files\Spaceadom`. It exists because some people expect an MSI. It is built from a custom template, `src-tauri/wix/main.wxs`, forked from the stock `tauri-bundler` file with exactly **four** changes, each marked `SPACEADOM CHANGE n` in the file. Read that file's own header before touching it. Two of the four are worth knowing here:

- The `INSTALLDIR` registry search was **deleted**. NSIS writes an HKCU key with its own per-user directory, and the stock template read that key — so a double-clicked `.msi` installed *into* `%LOCALAPPDATA%\Spaceadom` and registered the per-user app's files as its own. On **2026-09-04** an `msiexec /X` of that product then deleted the running app. The app vanished; only the config in Roaming survived. That is PROBLEM 244, and it is why section 10's hard rule about `msiexec` exists.
- `MajorUpgrade Schedule` was changed from `afterInstallInitialize` to `afterInstallExecute`, because the stock ordering queued a delete-on-reboot against the exe path, then wrote the *new* exe to that path, and the next restart deleted it.

Building the .msi writes an "installed the product" event into the Windows Application log — it is not an install. WiX's `light.exe` validates the package it just wrote by running it through the Windows Installer engine, five to ten seconds after the .msi's own mtime, every single build. The tell that separates it from a real install is the **transaction pair**: a genuine install or uninstall is bracketed by event 1040 "Beginning a Windows Installer transaction" and 1042 "Ending", naming either the .msi path or the ProductCode. The build-time events have neither.

`npm run store` — the 210 MB offline build

Overrides `webviewInstallMode` to `offlineInstaller`, because the Store forbids an installer that downloads bits when it runs, and the default `embedBootstrapper` does exactly that. Result: ~210 MB instead of ~5.6 MB. Right for the Store, wrong for a friend.

`poststore` renames it to `...-setup-STORE.exe`, drops it in `to-publish-in-microsoft-store/`, and deliberately leaves the normal installer path **empty**. So:

*After `npm run store`, **run** `npm run tauri build` **before installing locally**. Otherwise `install-real.cmd` has nothing to install — which is on purpose. An obvious failure beats silently installing the 210 MB build (PROBLEM 165).*

`npm run msix` — a different program

This packs the plain `npm run tauri build` release binary into an MSIX for the Store, using a hand-written `src-tauri/msix/AppxManifest.xml` and `MakeAppx` from the Windows SDK. Tauri v2 has no MSIX target.

It needs three Partner Center values in the gitignored `src-tauri/msix/identity.json` (copy `identity.example.json`). It leaves the package **unsigned** unless you pass `-Sign`, and it never installs anything — installing is an explicit act, never a side effect of a build.

Exit codes are three-way and meaningful: `0` packed and validated, `1` a real error, `2` the Windows SDK's `MakeAppx.exe` is missing (nothing wrong with the repo — install the SDK).

A packaged Spaceadom behaves differently in five places, and every difference is silent.

One probe decides them all — `packaged::is_packaged()`, logged at boot as the third line of every log. When packaged: the in-app updater is inert (the Store owns updates); autostart is the manifest's `startupTask`, not an HKCU Run value; config may get a one-time snapshot; the rival-install banner offers directions instead of a button; and the app does not touch `NotifyIconSettings` at all. AppData was measured **not** virtualised and HKCU **was**, on 2026-09-05.

Do not casually install the .msix on this machine — a packaged copy beside the NSIS one is two keyboard hooks fighting over the spacebar. It is allowed by exactly one procedure, in `CLAUDE.md`: kill the NSIS copy first, trust the test cert in `LocalMachine\TrustedPeople`, install, test, remove, relaunch the NSIS copy. Follow it or do not do it.

```
npm run portable
```

A zip with exactly three files: the exe, `portable.txt` (whose *presence* redirects all app data into `<exe dir>\data\`), and a read-me. Autostart is never registered and the updater is permanently inert. Three guards run before it packs — a size window, a version-stamp match, and a freshness check against `dist2` — each of which catches a real mistake that happened before.

7. Releasing to GitHub

Step 1 — bump three version numbers so they match

```
package.json          "version": "1.0.107"
src-tauri/tauri.conf.json "version": "1.0.107"
src-tauri/Cargo.toml   version = "1.0.107"
```

Careful in `Cargo.toml`: there is a second `version = "0.58"` further down — that is the `windows` crate pin, not the app.

This is not a convention, it is enforced. `release.yml`'s first real step reads all three plus the git tag and fails the whole job if any of the four disagree.

Step 2 — commit, tag, push

```
git add -A
git commit -m "1.0.107 - <what changed>"
git tag v1.0.107
git push
git push --tags
```

The tag is the trigger. Pushing the tag is what starts the release.

What the workflow does

`.github/workflows/release.yml`, on `windows-latest`, when a tag matching `v*` is pushed:

- Checks the three versions and the tag all agree — fails loudly otherwise.
- Writes the Sentry DSN from the `SENTRY_DSN` secret into `src-tauri/src/sentry_dsn.txt`. Fails loudly if it is not set.
- Checks both signing secrets are present. Fails loudly if either is blank.
- Node 20, Rust stable, cached. `npm ci`, then `npm run build`.
- `tauri-action` builds and signs both installers, and creates the GitHub release **as a draft**.
- Runs `scripts/write-updater-manifests.ps1` to produce `latest.json` and `latest-msi.json`, uploads them, then asserts all four expected assets are actually on the release.
- Builds and uploads the portable zip.
- Builds the `.msix` if the three MSIX secrets are set, and uploads it as a **workflow artifact only** — never as a release asset. The Store owns MSIX distribution; a `.msix` on the releases page

would hand somebody a second copy beside the `setup.exe` they already have.

- **Last step, and it must stay last:** flips the release from draft to published, then re-reads it to confirm the flip took.

Step 9 is last on purpose. Every installed copy polls `releases/latest/download/latest.json` once a day. Leaving the release a draft until everything is uploaded means users can never see a half-published release.

There is also a manual `workflow_dispatch` run that builds and signs everything and **publishes nothing** — a rehearsal. It used to accidentally create a real public release named after the branch. That is now structurally impossible, not just discouraged.

The secrets

Set in GitHub → repo → Settings → Secrets and variables → Actions.

Secret	What it is
TAURI_SIGNING_PRIVATE_KEY	The contents of <code>src-tauri/.tauri/spaceadom.key</code>
TAURI_SIGNING_PRIVATE_KEY_PASSWORD	The contents of <code>spaceadom.key.password.txt</code>
SENTRY_DSN	The crash-reporting endpoint
MSIX_IDENTITY_NAME, MSIX_IDENTITY_PUBLISHER, MSIX_PUBLISHER_DISPLAY_NAME	The three Partner Center values. Optional — the MSIX step skips if blank.
GITHUB_TOKEN	Provided by GitHub automatically.

If a release ever fails with `Wrong password` for that key, re-set the password secret with no trailing newline, from PowerShell:

```
gh secret set TAURI_SIGNING_PRIVATE_KEY_PASSWORD --body (Get-Content "src-tauri\.tauri\spaceadom.key.password.txt" -Raw).Trim()
```

A PowerShell pipe (`$pw | gh secret set ...`) appends a newline. That is a real suspicion about how these were originally set.

How updates reach users

Every installed copy runs a thread called `st-updater`. Fifteen seconds after launch — past the autostart settle — and then once a day, it fetches

`https://github.com/nur-arpon/Spaceadom/releases/latest/download/latest.json`. If the version there is newer, it downloads the installer, verifies the signature against the public key baked into the exe, and installs it **silently**: `setup.exe /S /UPDATE /R /ARGS --autostart`. The plugin exits the process; the NSIS installer relaunches it quietly.

The relaunch is the installer's `/R`, **not** `AppHandle::restart()` — `restart()` spawns the new process before the old one exits, and single-instance kills the newcomer (PROBLEM 233). Exit first, launch later.

An install is only ever updated by the kind of installer that made it. The app works out which at runtime: `uninstall.exe` beside the exe means NSIS; an HKLM `MsiExec /X` product registered for

the exe's folder means MSI; neither means never updated. NSIS installs read `latest.json`; MSI installs read `latest-msi.json`. Both are written by the same script from CI. Feeding an NSIS install the `.msi` is PROBLEM 129/244. **Do not "simplify" this to one manifest.**

The MSI leg is on, and it costs the user one UAC prompt per update — a per-machine `.msi` genuinely cannot install silently from a non-elevated process, because at `/qn` there is no UI through which the Installer service can ask for consent. `/passive` is the lowest level at which the prompt can appear at all. That was your decision on 2026-09-05: one prompt beats a published channel that is switched off. Setting `updater::MSI_AUTO_UPDATE = false` puts it back to "detected, never driven" as a one-word change. **No live MSI update has ever been observed.**

The escape hatch for a user is `"auto_update": false` in their `config.json`. There is no UI switch, by your decision.

Where things actually stand: the newest published release is **v1.0.95**. Local is 1.0.106. Nothing since 1.0.95 has been tagged or pushed, and the release secrets have therefore never been exercised in anger. The first real tag push is an experiment, not a routine — watch the Actions log.

8. Publishing to the Microsoft Store

The full instructions are in `to-publish-in-microsoft-store/START-HERE.md`, with `RUNBOOK.md` for the detail and `LISTING.md` for the text to paste. Read `START-HERE.md`; this is only the shape of it.

You are taking **Route B (MSIX)**. Route A — listing the `setup.exe` directly — is blocked on buying a code-signing certificate. Route B avoids that entirely because the Store re-signs the package itself.

The five things only you can do:

- **Clean this machine first (5 minutes).** Remove the local test certificate from `Cert:\LocalMachine\TrustedPeople` (thumbprint `710EB524F40B8233E6D485BCCC649839295BA0A1`), and delete the two leftover test files in `%APPDATA%\Spaceadom\`.
- **Create the Partner Center account, and start it today.** Go to <https://storedeveloper.microsoft.com> — that is the *only* entry point that gives the free individual tier; any other route shows the old paid flow. Individual developer, free. It requires government ID and a selfie, and there is no published turnaround time. **Everything else waits on this, so start it first.**
- **Reserve the name and copy three values.** Reserve `Spaceadom` (reservation holds three months). Then Product management → Product identity and copy `Package/Identity/Name`, `Package/Identity/Publisher` (a `CN=...` string) and `Package/Properties/PublisherDisplayName` into `src-tauri\msix\identity.json`.
- **Build the package.**

```
$env:CARGO_HOME="D:\RUST-DOWNLOADED-HERE\cargo"
$env:RUSTUP_HOME="D:\RUST-DOWNLOADED-HERE\rustup"
$env:PATH="D:\RUST-DOWNLOADED-HERE\cargo\bin;$env:PATH"
```

```
npm run build
```

```
npm run tauri build      # the plain release exe – NOT npm run store
npm run msix             # lays out, packs and validates
```

Expect `src-tauri\target\release\bundle\msix\Spaceadom_<v>_x64.msix` at about **10 MB**. If it comes out near 200 MB, you ran `npm run store` by mistake. Submit it **unsigned** — do not pass `-Sign`.

- **Submit through the wizard**, in this order: Pricing and availability (free, all markets) → Properties (Productivity, secondary Utilities + tools, plus the privacy/website/support URLs) → Age ratings (the IARC questionnaire; every answer No except "collects personal information", which is Yes, with the justification from `LISTING.md`) → Packages (upload the `.msix`; a `runFullTrust` notice here is expected, not a failure) → Store listings (paste from `LISTING.md`, upload the art per `assets/README.md`) → Submission options (paste the certification notes, which explain why a launcher needs a global keyboard hook — a human reads these) → Submit.

Then: security scan, automated technical compliance, and a human content review. Usually up to three business days, publishing about 15 minutes after it passes.

Traps in this route, all of which will cost you a rejection:

- Building with the placeholder identity still in `identity.json` (`LOCALTEST.Spaceadom`) produces a structurally valid package that Partner Center rejects at ingestion. The build warns loudly; read the warning.
- Do not upload the hero images with the tagline baked in. Only `Hero-1920x1080-textfree.png` is allowed in the wizard.
- Do not edit the copy of `PRIVACY.md` inside `to-publish-in-microsoft-store/` — it is overwritten by the build. The real one is at the repo root.
- `assets/screenshots/` holds five real 1920×1080 captures from the actual packaged app, taken on **your own profile** — personal app names and your taskbar are visible. Decide whether that is acceptable before you upload them.
- Nothing packaged has ever run on a machine with **no** unpackaged Spaceadom on it. Everything measured on 2026-09-05 was measured beside the NSIS install, and at least one finding depends on that. `SUBMIT-CHECKLIST.md` has the second-machine recipe.

9. Architecture, so you can reason about it

The whole app is one process, but you should think of it as **two worlds that must never touch each other directly**.

WORLD 1 – the hook thread

st-hook-supervisor

└─ hook_thread_main

 message pump

 kb_hook_proc

 ms_hook_proc

 ref_kb_hook_proc

└─ crossbeam bounded(256) →

WORLD 2 – everything else

the engine actor (async)

the two windows

the tray

the config

the updater thread

Anything the hook thread wants to say, it says by putting a small value on that channel. Anything the rest of the app wants the hook to know, it publishes into an atomic that the callback can read for free. That is the entire contract.

The hook thread and its message pump

A thread called `st-hook-supervisor` is spawned during startup. It runs `hook_thread_main` inside a `catch_unwind`, so if the hook body panics the supervisor logs it and restarts — capped at 5 restarts per 10 minutes so a crash cannot become a loop.

`hook_thread_main` does four things:

- Raises its own priority to `THREAD_PRIORITY_ABOVE_NORMAL`, so a heavy WebView2 render pass cannot starve the callback (PROBLEM 134).
- Calls `install_hooks()`, which installs three hooks: a **reference** keyboard hook (a do-nothing witness), the **real** keyboard hook, and the **mouse** hook — *in that order*, which matters enormously (law 5).
- Arms a `SetTimer` so `WM_TIMER` drives the watchdog.
- Blocks in `while GetMessageW(...)`.

That last line is the part people find surprising. A low-level hook callback is only ever called by Windows **on a thread that is pumping messages**. The loop looks like it does nothing; it is what makes the hooks fire at all. Delete it and the app goes silent with no error.

Why the callback may only touch atomics

`kb_hook_proc` runs on the OS input path. Every key on the machine waits for it to return. Windows measures how long it takes, against `LowLevelHooksTimeout` in `HKCU\Control Panel\Desktop` (1000 ms by default), and if it overruns, Windows **stops calling it and leaves the handle valid**. No message. No error. No return code. `UnhookWindowsHookEx` on it still succeeds.

So the callback is allowed to:

- read and write `AtomicBool` / `AtomicU32` / `AtomicU64`;
- call `try_send` on the channel (non-blocking — a full channel just bumps a `DROPPED_EVENTS` counter rather than stalling).

And it is forbidden to:

- call `log::` anything. Log writes go to disk. **PROBLEM 58: logging inside the hook killed the hook**. There is a comment banner in `hook/mod.rs` saying `NO log:: CALLS BEYOND THIS POINT` — obey it.
- touch Tauri, COM, or the webview;
- take any lock that could block;
- read state that needs a Win32 round-trip. Anything like that is polled by a separate watcher thread and republished into an atomic.

Diagnostics are collected as lock-free counters and drained later, off the callback, by `drain_hook_diagnostics()`.

The channel

```
crossbeam_channel::bounded::<hook::HookEvent>(256)
```

Created once in `lib.rs`. The sender goes to the hook thread; the receiver goes to the engine. `HookEvent` is small:

```
SpaceDown · SpaceUp { modifier_fired } · KeyCombo(KeyCombo) · WheelUp · WheelDown ·  
PointerActivate(char) · OwnWindowSpaceDown
```

and `KeyCombo` is `Alpha(char)` plus the named specials: `Special(String)`, `Escape`, `Backtick`, `Comma`, `RightAlt`, `UpArrow`, `DownArrow`, `Period`, `Backspace`, `Tab`.

The engine actor

`engine::start_engine` spawns onto Tauri's async runtime. Its loop uses `spawn_blocking(move || rx.recv())` to pull from the blocking crossbeam receiver without tying up an async worker, and then dispatches **each event in its own task** — so a panic while handling one keypress is caught and logged instead of killing the actor.

`dispatch()` matches the event and calls a handler: `handle_alpha`, `handle_special`, `handle_boss_key`, `handle_pip`, `handle_force_close`, `handle_focus`, `handle_profile_cycle`, `handle_bypass_toggle`, and the opacity actions for the wheel events. The actions themselves live in `engine/actions/` — `smart_cascade`, `boss_key`, `pip`, `opacity`, `focus_engine`.

The journey of one keypress

You hold Space, tap `s`, and Spotify appears. Here is every step, so that when one of them breaks you know which one to look at.

- **Space goes down.** Windows calls every `WH_KEYBOARD_LL` hook on the machine, newest-installed first. Ours is `kb_hook_proc`.
- The callback stamps `LAST_KB_EVENT` and `LAST_KB_CALLBACK`, bumps `KB_EVENTS_SEEN`, and checks `dwExtraInfo` against our own cookie `0x7A7A7A7A` to see whether we injected this ourselves. We did not, so it is a real key.
- `MODIFIER_ACTIVE` is false, no Ctrl/Alt/Win is held, so: set `MODIFIER_ACTIVE = true`, `SPACE_ABORTED = false`, stamp `SPACE_DOWN_TS`, send `HookEvent::SpaceDown`, and **return** `LRESULT(1)` — which swallows the keystroke. The OS never sees a space yet.
- The engine receives `SpaceDown`, makes a cancel channel, reads `guide_hud_delay_ms` from config, and starts a timer: if Space is still down when it fires, show the ring.
- **You keep holding.** Windows auto-repeats the Space-down. The callback sees `MODIFIER_ACTIVE` is already true, bumps `SPACE_REPEATS`, and sends nothing. That is how the app never leaks repeated spaces.

- **You tap `s`.** The callback sees a key-down with `MODIFIER_ACTIVE` true, so it enters the combo branch. Three gates run first: the stuck-modifier failsafe (`MAX_MODIFIER_HOLD_MS`); the **rollover window** — if less than `ROLLOVER_MS` (default ~200 ms) has passed since Space went down, you were typing fast, not commanding, so it aborts and lets both keys through; and a check that a real OS shortcut overlapping the hold (Win+Shift+S) wins.
- Past the gates, `s` classifies as `KeyCombo::Alpha('s')`. Set `SPACE_ABORTED = true`, send the event, return `LRESULT(1)` so no `s` leaks into whatever app you were in.
- The engine cancels the HUD — with `action_pending = true`, which keeps the overlay *window* up so the toast has somewhere to land (PROBLEM 135) — and calls `handle_alpha('s', ...)`.
- `handle_alpha` reads the active profile out of `Arc<RwLock<AppConfig>>`, looks up "s" in that profile's *bindings*, and falls back to the Founders profile's binding if this profile has none.
- It calls `smart_cascade`, which **first tries to focus or minimise**. It looks up a cached `HWND` for the target, and revalidates it — `IsWindow`, the owning process, the window class — because Windows recycles handles and a stale one is how you minimise the taskbar.
- Already focused and not minimised → `ShowWindow(SW_MINIMIZE)`.
- Running but behind → `ShowWindow(SW_RESTORE) + force_foreground`.
- Not running → fall through.
- Not running, so `shell_launch` calls `ShellExecuteExW` (unelevated), `AllowSetForegroundWindow` so the new window may take focus, and `raise_after_launch` restores and foregrounds the window once it appears. Launching *without* that last step is a real bug that shipped — the app started and nothing ever came to the front (PROBLEM 170).
- The outcome (`Primary / Fallback / Failed`) picks a toast, and `show_toast` does a **global** `emit("toast-notification", ...)`. Only the overlay page listens.
- **You release Space.** The callback clears `MODIFIER_ACTIVE`, sees `SPACE_ABORTED` is true — a combo fired — and therefore does **not** inject a literal space. Sends `SpaceUp { modifier_fired: true }`, returns `LRESULT(1)`.

If you had released without tapping anything, step 13 would have found `SPACE_ABORTED` false and injected a real space with our cookie on it, and step 2 of the *next* callback would recognise the cookie and pass it straight through.

The two windows

Loads	<code>index.html</code>	<code>overlay.html</code>
Look	1220×880, decorated, resizable	transparent, undecorated, no shadow
Behaviour	closes to tray	always-on-top, click-through, never focused
Size	fixed by the user	sized on demand (e.g. 600×460)
Gets	config, profiles, the whole dashboard	the ring and every toast

Both are declared in `tauri.conf.json` with `create: false` and built by `create_app_windows()` in `lib.rs`, which also has a hand-written fallback path for when `WebView2` fails to attach on a cold boot.

Rules that were paid for:

- **A window not listed in `src-tauri/capabilities/default.json` is deaf** — every `listen()` in it rejects silently.
- **Global `emit` + a single listener is the only arrangement that works here.** `emit_to` has never delivered, in any combination tried.
- The overlay is **centred** while the ring owns it and **bottom-centred** for toasts. Two different placements, deliberately not unified.
- Transparency is conditional, not banned: a *fullscreen* transparent webview composes zero pixels on this machine; the *small on-demand* one works.
- `backdrop-filter` is banned. `filter: blur()` has a size limit — `340×150` at `blur(22px)` works; `560×320` at `blur(34px)` killed the entire window (PROBLEM 37). Bake softness into gradient stops instead.
- Show the window *before* emitting content. Hidden webviews do not paint.
- If `set_ignore_cursor_events` fails, the code hides the overlay. Fail closed: an always-on-top window that eats clicks means the user cannot click anything at all.

The config

<code>%APPDATA%\Spaceadom\config.json</code>	(normal and packaged installs)
<code><exe_dir>\data\config.json</code>	(portable, when <code>portable.txt</code> is present)

Shape, simplified:

```
AppConfig
  active_profile: String
  profiles: Vec<Profile>
    Profile { name, bindings: BTreeMap<String, KeyBinding>, emoji }
      KeyBinding { app, web_url, label, icon_override, ... }
  rollover_ms, guide_hud_delay_ms, run_at_startup, theme, dark_mode,
  overlay_compositing, opacity_floor_pct, fullscreen_allowlist, ...
```

Held as `Arc<RwLock<AppConfig>>`. `config::save` writes the file **and** republishes several hook-facing atomics (bound specials, excluded apps, the pointer-HUD flag) so a settings change takes effect without a restart.

`set_active_profile` **saves by itself**. Do not also call `persistConfig()` from the frontend — that was the double-save bug.

`%APPDATA%\Spaceadom\` also holds `debug.log`, `picker-cache.json`, `boot-attempts.json` (the safe-mode counter), `reports\` (diagnostics zips) and `rollback\` (archived installers for the in-app rollback).

Boot order in `lib.rs`

- Elevation check — deliberately a no-op. **The app does not elevate** (PROBLEM 61).

- Logger init. Must come before any `log::` call.
- A background thread registers autostart, scans for a rival install, and removes legacy Run entries. It does not block boot.
- Load config; republish specials, exclusions, pointer-HUD and telemetry consent into atomics. Set the `WebView2 --disable-gpu` argument here if compositing is set to software — it must happen before `WebView2` exists.
- Create the `bounded(256)` channel.
- Create the shared icon cache.
- Build the Tauri app and enter `setup()`. Inside it, in order: note this launch survived (the safe-mode counter); **spawn the hook thread** — unless this is a safe-mode launch, in which case the sender is parked and no hook is ever installed; start the fullscreen, app-exceptions and pointer watchers; construct `EngineState` and start the engine; build the tray; arm the `WM_ENDSESSION` guard.
- **Then** create the windows. On an autostart launch this is deliberately delayed a few seconds (PROBLEM 215) — so Space+key works immediately at logon, just without the ring for a moment.
- Start the `st-updater` thread.

How the UI talks to Rust

Two directions, both standard Tauri.

Frontend asks Rust to do something:

```
await invoke("save_config", { newConfig: appConfig });
```

Rust tells the frontend something happened:

```
await listen("hook-status-update", async () => {
  const s = await invoke<HookStatus>("get_hook_status");
  applyHookState(s.installed);
});
```

There are about 78 commands. The ones you will meet most: `get_config`, `save_config`, `get_profiles`, `set_active_profile`, `get_hook_status`, `get_hook_health`, `reinstall_hook`, `toggle_bypass`, `overlay_fit`, `overlay_fit_hud`, `overlay_toasts_done`, `publish_hud_chips`, `list_start_menu_apps`, `extract_icon_cmd`, `get_about_info`, `check_for_updates_now`, `rollback_available`, `rollback_to_previous`, `set_startup_enabled`, `build_diagnostics_bundle`, and the three own-window fallback commands `own_window_space_down` / `own_window_key` / `own_window_space_up`.

Every new command must be added to the `tauri::generate_handler!` list in `lib.rs`. A command that exists in `commands.rs` but is not in that list is invisible — the frontend's `invoke` just rejects. That has happened twice.

10. The laws, and why each exists

Every one of these was written after something broke. The reason is attached so you can tell when a law genuinely applies and when it does not.

The seven keyboard-hook laws

1. Filter injected input ONLY by our own `dwExtraInfo` cookie `0x7A7A7A7A` — never by `LLKHF_INJECTED`. Blanket-ignoring injected input kills the app for anyone using AutoHotkey, a macro keyboard, the on-screen keyboard, or Remote Desktop. The July builds were silently dead for all of those users.

2. If you need two keys in a guaranteed order, send them in ONE `SendInput` batch. `SendInput` followed by `CallNextHookEx` does not preserve order. That produced the `hte-for-the` bug: type fast enough and your letters come out swapped.

3. `GetAsyncKeyState` reports a key we SUPPRESS as UP. So you can never build a failsafe on it for a key you are hiding. The stuck-modifier detection uses a timestamp latch instead. This also means combos cannot be tested by injection the naive way.

4. Pass Space through when Ctrl, Alt or Win is physically held. Those are OS features — IME switching, IDE autocomplete, the window menu. Shift is deliberately excluded from that list.

5. THE REFERENCE HOOK GOES IN FIRST. DO NOT REORDER `install_hooks()`. DO NOT "SIMPLIFY" IT. This is the one an AI will try to tidy, so understand it properly. Windows calls low-level hooks **newest-first**: `SetWindowsHookExW` puts each new hook at the *head* of the chain. `CallNextHookEx` is synchronous, so the newest hook's "am I too slow?" clock has to cover its own body **plus the whole chain running underneath it**. A do-nothing witness hook installed *last* is therefore the *slowest-looking* hook in the chain — and Windows evicts the slowest first. So the witness died, and a dead witness can only ever report "all quiet", which the deaf-detector read as permanent health while the watchdog kept tearing down a hook that was working perfectly. Measured: **514 false teardowns in one 3.8-hour session**. Fixed in 1.0.96 on 2026-09-04 by installing the witness first, so it lands at the *back* of the chain. *What you see when this breaks*: shortcuts randomly stop mid-hold, especially while the dashboard is focused. *The one-line test*: `grep "genuinely fired" debug.log | tail` — the (N total) counter must keep climbing — and `grep -c "WATCHDOG - " debug.log` must stay near 0. Full account: `docs/IF-SHORTCUTS-DIE-AGAIN.md`.

6. THE RING MUST SHOW OVER OUR OWN WINDOW, and you must check it before every ship. Hold Space with the dashboard focused, and confirm the log has a `hold start (hold #N) ...` over own window line **followed by** `guide_hud: shown over own window`, after the new build's version banner. Why this is a law: 1.0.101 and 1.0.102 both shipped "proved" on 2026-09-05 while every Space held inside the dashboard was invisible to the keyboard hook (PROBLEM 257 — the mouse hook on the same thread fired 2,705 times in a minute in which both keyboard hooks fired 0). *And there are now two witnesses, so read which one fired*:

What fired	The pair to grep
The keyboard hook (this law's proof)	hold start (hold # + a digit, then guide_hud: shown over own window
The page-side fallback (PROBLEM 259)	own-window fallback: then guide_hud: shown over own window

A ring you saw inside the dashboard is **no longer evidence that the hook is alive**. If only the fallback line is there, the report says UNPROVEN.

7. A TIMED-OUT keyboard hook stays installed and is never called again — and a PROVEN-deaf verdict must never be blocked by a cooldown. Two facts. (a) When the callback overruns `LowLevelHooksTimeout`, Windows stops calling it and leaves the handle valid. From inside the process, an evicted hook and a hook nobody has typed into are the same observation. (b) The mouse hook is a **separate hook with its own timeout record**, even on the same thread from the same pump — so it keeps firing at 30–60 Hz while the keyboard hooks are dead. The only cure a process has is to change the chain: unhook and install afresh. Three consequences that are law:

- **A live mouse callback may never veto a keyboard-deaf verdict.** It proves the *pump*, not the hook.
- **Cooldowns throttle unevidenced alarms only.** A verdict proven from callback-only clocks bypasses every throttle. Waiting is strictly worse than repairing when the app is provably deaf.
- **A repair is an unhook plus a fresh install, and the log prints old → new handle values to prove it.** A handle that did not change means the install failed. Never argue about whether a repair happened — read the handles. Cost of not having this: 130 seconds of provable deafness on 2026-09-07 during which the watchdog printed *nothing*. *The signature to grep:*
`hook liveness split - primary_real:0 ... reference:0 mouse:<nonzero>.`

And the eighth thing, which is really a corollary (PROBLEM 262, 2026-09-07): *an instrument that can only be read by the thing that has failed is not an instrument*. A Space-hold latch was bounded by three timers that all lived on the keyboard callback — the one component a deaf hook is defined by never running. So the latch could not be cleared, the fallback refused to raise a ring while it was set, and the ring stopped appearing at all until a restart. Ask of every timeout you write: **what resets this, and can that happen while the failure is in progress?**

Window and overlay rules

- A window must be in `capabilities/default.json` or it is deaf.
- Global `emit`, single listener. `emit_to` does not work here.
- Overlay centred for the ring, bottom-centred for toasts. Do not unify.
- Small transparent window: fine. Fullscreen transparent window: composes nothing.
- `backdrop-filter`: banned. `filter`: `blur()`: size-limited (PROBLEM 37).
- **Never remove the overlay's logging.** `overlay_fit` and `overlay_fit_hud` log the requested size, the monitor, and the resulting size, position and visibility. Without it, a wrong size, a wrong position and a window that never moved are indistinguishable.

- **That covers `hide()` and `show()` too (PROBLEM 135).** A silent `win.hide()` hid the overlay 500–1000 ms before each toast existed, and three builds of animation work played out invisibly while in-page instrumentation reported perfect health — **because a page cannot observe that its own window is hidden.** Any call that changes what the user can see must say so in the log. If a visual is missing and everything measurable inside the page is fine, **suspect the window before the page.**
- Undecorated Windows 11 windows draw a 1px DWM border that reads as a box. Clear it with `DWMWA_BORDER_COLOR = 0xFFFFFFFF` and `DWMWCP_DONOTROUND`.
- To diagnose a visual artifact, **sample the pixels** (`Bitmap.GetPixel` over a screenshot) rather than eyeballing a zoom. That is what proved the overlay interior really was transparent and that the leftover "box" was a single 1px line — two rounds of guesswork replaced by one measurement.
- CSP `style-src` needs `'unsafe-inline'` or every `style="..."` attribute dies silently. No external hosts, ever; fonts are bundled.

Design rules

The design files under `..\design\` are **specifications, not suggestions**. Every value in them is literal — transcribe, never paraphrase.

- **Radii:** 13px for keys and cards, 16px for containers, 999px for everything interactive. The tokens are `--radius-md`, `--radius-lg`, `--radius-full` in `src/styles/design-system.css`.
- **Motion:** `--dur-micro` 180ms, `--dur-standard` 320ms, `--dur-hero` 560ms. Entrances use `--ease-spring`, reflow and hovers use `--ease-out`, and **exits run at roughly 65% of the entrance time with `--ease-in`**. A UI that leaves as slowly as it arrives feels sticky.
- **Shadows** are warm brown `rgba(90,60,30,...)` in the Earthy palette and black-tinted in Nocturne. Never pure black on cream.
- **Nothing may assume a fixed width around an app name.** Names vary wildly.
- `prefers-reduced-motion` **renders final states** — not slower animation, the finished frame. There is also a `.lite-scene` mode that halves the storm's blur radii on weak machines.
- **One theme setting drives everything.** `body.nocturne` on the dashboard *and* the overlay. The overlay learns about it two ways and needs both: `save_config` re-emits `theme-changed` from Rust, and `overlay.ts` seeds itself from `get_config` on load — because an event that only fires on *change* leaves a freshly-opened overlay in the wrong palette.

A control that does nothing is worse than a missing control

If a switch appears in Settings, it must do something the moment it is switched. The rule this generates is an ordering rule: **add the Rust command first, then the toggle**. "Run at startup" was the original offender; it now has `set_startup_enabled`, which flips the HKCU Run value and persists the config in one call, and the toggle ships.

The related rule for a control that *can't* work right now: make it **inert and say who is holding it** — greyed out with a sentence naming the reason — rather than letting it flip back on its own. That is

what a packaged build's startup row does, because Windows owns that setting inside a package and `RequestEnableAsync` is documented to refuse to override a user who switched the app off in Task Manager.

The documentation rule

Every solved problem gets TWO entries, and neither is optional. Your own reason, in your words: *without documentation an AI has to start from scratch, and that costs a huge number of tokens*. Documentation here is a deliverable, not a courtesy.

- `PROJECT_STATUS.md` — dated, signed, newest at top, append-only. Never delete an entry. Say what happened **and under what condition it failed**.
- `V14_FIXES_AND_CODE.md` — the technical record, in this exact shape: **Symptom** → **Root cause** → **Exact file** → **The actual code (paste it, before and after where it helps)** → **How it was verified**. Plus a "generalise this" line when the bug has a class.

The test for a good entry: **could another person, or another AI, apply the fix from this file alone, without opening the codebase to search for it?** If not, it is not finished.

Two more habits inside that rule, both of which have already paid for themselves:

- **Record the class, not just the instance.** "An ID rule that sets `display` needs its own `#id[hidden]` companion" is reusable. "The profile popover was open" is not.
- **Record the CONDITION a thing failed under, and how to re-test it.** A note that just said "transparent windows render nothing" cost a later session two failed redesigns, because the truth was "*fullscreen* transparent windows render nothing; small on-demand ones are fine".
- **When a fix turns out to be unnecessary, or a diagnosis was wrong, write that down too.** The "measurement traps" sections exist because two false bug reports were nearly filed. That is worth more than a clean story.

Hard rules that are not about code

- `msiexec /X` **can delete the live app**. An MSI product's file list may point anywhere. On **2026-09-04** a banner offered to remove a "leftover" registry entry, `repair()` ran `msiexec /X{GUID}`, Restart Manager failed to close the running app, and Windows Installer deleted the product's registered files anyway — the live `spaceadom.exe`. The app vanished; only the Roaming config survived. **A "leftover entry" is removed from the REGISTRY ONLY, never through the Installer.** Generalise: *a removal that trusts a registry entry's own description of what it owns is a removal aimed by the thing being removed*.
- **Never delete, rename or overwrite a file you did not create** — stale- looking or not, `.tmp/.bak/.old` or not. A `PROJECT_STATUS.md.tmp` was deleted by an agent on 2026-09-04 on exactly that reasoning; the file it looked like a leftover of was 580 KB of append-only log, and only luck decided whether the `.tmp` was garbage or the only copy of an in-flight write. **A prepend is an overwrite** — on 2026-09-07 a one-liner that read a file and wrote it back in the same statement had its `read` fail and its `write` succeed, and `PROJECT_STATUS.md` went from 817,401 bytes to 6,121. The safe form is: write the new head to a temp file, append the old file to it, move it into place.

- Do not modify `D:\SpaceToggle-July_Revisit_2026`, `D:\GITHUB PROJECT` or `D:\Neon` — read-only reference.
- Never remove a `CORE_AIM.md` feature to make something build.
- No CDN anything. The app must work fully offline.
- Your laptop panel is 2560×1600 at 150% (1707×1067 logical) and you plug a second display in and out through the day. **Display changes are routine here, not an edge case.** The Guide HUD stays primary-monitor-only by your explicit decision — do not "fix" that without deciding to.

11. How to debug anything

This is a method, not a checklist. Follow it in order.

Step 0 — read the log FIRST

```
notepad "$env:APPDATA\Spaceadom\debug.log"
```

Not the code. The log. This is written down because it had to be pointed out once, and the log then found two causes that reading the code could not.

The format is:

```
2026-09-07 12:17:35.178 [WARN] space_toggle_os_lib::hook — hook: KEYBOARD DEAF, PROVEN ...
```

Timestamp, level, module, then a message written to be read by a person. Most of them explain themselves — the app deliberately logs sentences, not codes.

The third line of every log is the packaged-identity probe. The build banner (`Spaceadom build – version ...`) marks the start of each run: **everything you assert about "this build" must come from after that line.**

The greps that answer the common questions

Run these from Git Bash, or use `Select-String` in PowerShell.

Question	Grep	
Is the hook alive?	<code>`grep "hook liveness split" debug.log \</code>	<code>tail`</code>
Is the witness hook alive?	<code>`grep "genuinely fired" debug.log \</code>	<code>tail – the (N total)` must climb</code>
Are there false teardowns?	<code>grep -c "WATCHDOG – "</code> <code>debug.log — should be near 0</code>	
Was the hook proven dead?	<code>grep "KEYBOARD DEAF, PROVEN"</code> <code>debug.log</code>	
Did a real hold reach the hook?	<code>grep "hold start (hold #"</code> <code>debug.log</code>	

Question	Grep	
Did the page-side fallback fire instead?	grep "own-window fallback:" debug.log	
Did the ring appear over our own window?	grep "guide_hud: shown over own window" debug.log	
Is the app in safe mode?	grep safe-mode debug.log	
What did the engine decide?	`grep "engine:" debug.log \	tail -40`
Where did the overlay go?	`grep "overlay_fit" debug.log \	tail`
Did a ring get stuck?	grep "WATCHDOG alarm confirmed, but a Space hold is LIVE" debug.log — more than one per hold is a bug	
Did the updater run?	`grep "updater:" debug.log \	tail`

hook liveness split is the single most useful line in the file. It reports four callback-only counters for one 60-second window:

```
primary_real:0 primary_injected:0 reference:0 mouse:217 in the last 60s
```

mouse above 0 with primary_real and reference both 0 is a **named signature**: timeout eviction of the keyboard hooks with a healthy pump. It is not "nobody typed", and it is not a wedged thread.

Step 1 — is that zero real?

Before you believe a zero, prove the check can produce a non-zero.

The rule: **a check that cannot produce a negative result is not a check**. Three real ways it has failed here:

- **An encoding.** A pattern with . standing in for an em dash reported 0 alarms against a file `grep -c` found 2,312 in. Windows PowerShell 5.1 reads a BOM-less UTF-8 file as ANSI, so the em dash arrives as *three* characters and a one-character wildcard can never span it. **The pattern could not match on any input.**
- **A self-match.** A log line that tells you what to grep for **becomes a hit for that grep**. The PROBLEM 259 fallback line quotes the advice `hold start (hold #N) ... over own window`, so a bare `hold start` matches it. A real hook hold reads `hold start (hold #` followed by a **digit**; the quoted advice has the letter **N**. That was the difference between 2 and 0.
- **A drained counter.** `hook diagnostics` swaps its counters to zero every 60 seconds. A number printed beside the word "session" must read a counter nothing drains, or you get `repair #1 this session` printed three times and conclude the repair never happened.

So: **put a control beside every count**. Run the same pattern over the whole log, all boots, where you *know* there are hits, and print both numbers. A zero next to a healthy control is evidence. A zero on its own is a shrug.

Step 2 — write a probe with a positive control

When you need to measure something, the probe must first demonstrate it can detect the thing at all.

- Testing an injection harness? Run it with the app **stopped** first. If it still fails, the harness is broken, not the app.
- Testing a marker scan? Scan a marker you *know* is in the file.
- Testing whether a file is stale? Read the same path from two different contexts and compare (the differential rule, section 5).

If the positive control fails, the run is **VOID**. Say so and stop. Do not report a negative from a probe that never demonstrated a positive.

Step 3 — bisect using `all-versions/`

`all-versions/` holds every installer ever built — over 200 of them, back to `SpaceToggle V14_14.0.0`. That is a working time machine.

To find when a behaviour broke:

```
taskkill /IM spaceadom.exe /F
D:\Claude-Projects\SpaceToggle-V14\all-versions\Spaceadom_1.0.99_x64-setup.exe /S
Start-Process "$env:LOCALAPPDATA\Spaceadom\spaceadom.exe"
```

Use it, live with it for a while, then halve the interval. **Back your config up first** —

`%APPDATA%\Spaceadom\config.json` — because an older build may not understand a newer config field. There are already `_config-live-copy-*` snapshots at the repo root from previous rounds of exactly this.

Two cautions:

- Only use the **NSIS** `setup.exe` files for this. Do not double-click the `.msi` files in that folder — that is what created the registration behind PROBLEM 244.
- Note the *last good* and *first bad* versions in `PROJECT_STATUS.md` with their timestamps. "Last good 2026-09-05 14:11, first bad 15:35" is the single most useful sentence you can write about an intermittent bug.

Step 4 — the honesty rule for intermittent bugs

An intermittent bug proved fixed by one good hour is not fixed.

If a fault appeared roughly twice an hour, an hour without it is barely evidence. Say what the rate was before, what the rate is now, over what duration. `21.0 deaf-minutes per 100 active minutes` is a claim. "It seems better" is not.

And: **never report something as fixed, working or verified unless you observed it working.** Label untested things untested. That costs you nothing and it is the single habit that keeps this project trustworthy to yourself six months from now.

12. How to add a feature without breaking things

The order below is not arbitrary. Each step exists because doing it later caused a specific problem.

1. Read first. `CLAUDE.md` for the rules that touch what you are about to change, then search `V14_FIXES_AND_CODE.md` for the area. If there is a **PROBLEM** entry about the file you are opening, read it. It exists so you do not have to search.

2. Add the Rust command first, then the UI. A control that does nothing is worse than a missing control. Write the command in `commands.rs`, **add it to the** `generate_handler!` **list in** `lib.rs` (forgetting this is a real, repeated bug — the command exists and `invoke` just rejects), and check it works before you draw anything.

3. Keep new frontend modules LEAF modules. A module that imports from `main.ts` cannot be rendered by the preview harness, and then the only way to look at your component is a full Rust build. Everything under `src/components/` is a leaf on purpose (that is **PROBLEM 148**).

4. Run the gates. All four, every time:

```
cd D:\Claude-Projects\SpaceToggle-V14\src-tauri
cargo test --lib
cargo clippy --all-targets
cd D:\Claude-Projects\SpaceToggle-V14
npx tsc --noEmit
npm run build
```

Expect **0 failures, 0 clippy warnings, 0 type errors**. The test count is around 556 and should only go up. If it went down, you deleted a test — find out why.

Add a test when the logic is pure and the branch is one a user only reaches after something has already gone wrong. Recovery branches that have never been executed are how **PROBLEM 118** happened.

5. Hand-test what the gates cannot reach. Anything touching the keyboard hook, an overlay window, or the installer needs a human on the real machine. The gates cannot press Space.

6. Write the two documentation entries (section 10). Do this *before* the build, while you still remember the condition it failed under.

7. Then build, install, and prove (section 5). Bump the three versions, build signed from PowerShell, check the new marker is `True` in the fresh exe and `False` in the old installed one, install through `explorer.exe`, read `install-check.txt`, and hold Space.

13. Things you must not lose

Losing any of these costs you something you cannot rebuild.

1. The updater signing key — the worst one

```
src-tauri\src\spaceadom.key           the private key
src-tauri\src\spaceadom.key.password.txt its password
src-tauri\src\spaceadom.key.pub        the public half
```

The whole `src-tauri\src` directory is gitignored, so **it is not in your repo and it is not on GitHub**. If your D: drive dies, it is gone.

The **public** half of that key is compiled into every copy of Spaceadom ever installed, as `plugins.updater.pubkey` in `tauri.conf.json`. Every installed copy checks every update against it. So:

If you lose the private key, no existing install can ever accept an update again — forever. Not "you have to re-sign". They are stranded. The only way back is to persuade every user to manually download and run a new installer built with a new key.

Back it up today, somewhere that is not this repo and not this machine: a password manager, an encrypted archive on a USB stick you keep elsewhere, a private cloud vault. Back up **both** the key and the password file — one without the other is useless. Key ID `9548E059051C68CB`, if you ever need to confirm you have the right one; both shipped `.sig` files decode to it, and `spaceadom.key.pub` is byte-identical to what is in `tauri.conf.json`.

Do not regenerate it. There has already been one day where the password looked wrong and regenerating was tempting. The password was correct; Bash was mangling it. Regenerating would have stranded every install.

2. The Sentry DSN

`src-tauri\src\sentry_dsn.txt`, gitignored. It is the address crash reports go to. Safe to ship inside the built app (a DSN can only *send*), but never in git history. The example file beside it explains the setup. Without it, crash reporting is decoration — every report goes nowhere, silently.

3. The GitHub Actions secrets

`TAURI_SIGNING_PRIVATE_KEY`, `TAURI_SIGNING_PRIVATE_KEY_PASSWORD`, `SENTRY_DSN`, and optionally the three `MSIX_*` values. They live in GitHub → Settings → Secrets and variables → Actions. **You cannot read them back** — you can only overwrite them. So the local files above are the only copies.

4. The MSIX identity

`src-tauri\msix\identity.json` — the three Partner Center values. Not secret, but a nuisance to re-derive, and until you have the Partner Center account it does not exist yet.

5. The documentation itself

`V14_FIXES_AND_CODE.md` (1.7 MB) and `PROJECT_STATUS.md` (790 KB) are the most valuable files in the repo after the source. They are the reason a new session, human or AI, can be useful in ten minutes rather than three hours.

`PROJECT_STATUS.md` **has already been destroyed once**, on 2026-09-07, by a one-liner that read and wrote it in the same statement. It was rebuilt from a week-old git copy plus session transcripts,

and the recovery notice at the top of the file is honest about what may still be missing.

Commit them. They are the one thing in this repo that git protects and nothing else does. `git push` after every session that wrote to them.

14. DOOMSDAY PROTOCOL — continuing without Claude Code

This is the section to re-read on a bad day.

Everything before this assumed you could ask something for help. This section assumes you cannot. No Claude Code, no session that already knows the project, nobody to hand the log to. Just you, this repo, and a machine.

The good news is real and worth saying first: **nothing about this project depends on an AI being present.** The code is on your disk. Every problem that was ever solved is written down with its root cause and its fix. The gates run from a terminal. The installer proves itself. The parts that needed a person have always needed a person — you are the one who held Space and said "the ring did not appear," and that has never been automatable.

What you lose without an assistant is speed, not capability. So this section is about getting the speed back: where to start reading, how to get another AI to be useful instead of confident, what to do in the first five minutes of a breakage, and how to undo a bad day in two minutes.

14.1 What is already safe, right now

Before anything else: **the irreplaceable material is already backed up.** As of 2026-09-07 it lives on the pen drive:

```
E:\Spaceadom-Backup-2026-09-07\  
  01-SECRETS-DO-NOT-SHARE\    <- the only folder that cannot be recreated  
    .tauri\spaceadom.key  
    .tauri\spaceadom.key.password.txt  
    .tauri\spaceadom.key.pub  
    sentry_dsn.txt  
    identity.json  
    READ-THIS-FIRST.txt  
  02-SOURCE\                  <- the whole repo + spaceadom-history.bundle  
  03-INSTALLERS\               <- the recent setup.exe / .msi / .msix + .sig files  
  04-MY-SETTINGS\              <- your live config.json and debug.log  
  README.txt                   <- the restore procedure, five steps
```

Section 13 told you what each of those things is and what losing it costs. This is where it *is*. Two habits keep it true:

- **Refresh 01-SECRETS-DO-NOT-SHARE never** — it does not change. The signing key is generated once and lives forever. If you ever find yourself regenerating it, stop and re-read section 13 first.

- **Refresh 02-SOURCE and 04-MY-SETTINGS after any session that mattered.** A month-old source backup plus a current git push is fine; a month-old `config.json` is a month of your own key bindings gone.

Check the pen drive is still readable about once a month. A backup nobody has ever read back is a hope, not a backup.

```
Get-ChildItem "E:\Spaceadom-Backup-2026-09-07" -Recurse -File |
Measure-Object -Property Length -Sum
```

If that errors, the drive is the problem — copy the folder somewhere else today.

14.2 How to orient yourself, alone

You will open this repo one day with no memory of what you were doing. There are 25 markdown files at the root and 1.7 MB of fix notes. That is a lot to face cold. Do it in this order and it takes ten minutes, not three hours.

1. Read `CLAUDE.md`. All of it, every time you come back after a break. It is about 56 KB and it is the law: identity, build commands, the seven hook laws, the window rules, the testing laws, the hard rules. It is written to be re-read, not skimmed once. Everything else in the repo is subordinate to it — including this guide. **If this guide and `CLAUDE.md` ever disagree, `CLAUDE.md` is right and this file is stale.**

2. Read the newest entry in `PROJECT_STATUS.md`. It is newest-at-top, so that is the first entry in the file after the header. It tells you what the last session did, what was left half-done, and under what condition something failed. This is the single most useful minute you will spend.

3. Now find the **PROBLEM entry for whatever you are about to touch.** Do not read `V14_FIXES_AND_CODE.md` front to back — it is 1.7 MB and it is a reference, not a book. Search it:

```
Select-String -Path V14_FIXES_AND_CODE.md -Pattern 'overlay' -Context 0,2 |
Select-Object -First 40
```

Or from Git Bash, which is faster for this:

```
grep -n "^## PROBLEM" V14_FIXES_AND_CODE.md | tail -40
grep -n -i "guide hud" V14_FIXES_AND_CODE.md | head -30
```

Search by symptom, not by file name. The entries are written around what the user saw. "the ring did not appear", "shortcuts died mid-hold", "the installer exited 0", "the toast never showed" — those are the words that will find the entry.

4. Only then open the code. Ninety percent of the time the entry you found names the exact file and function and pastes the code, and you are editing with the reasoning already in your head instead of reconstructing it.

That is the whole method. `CLAUDE.md` → newest `PROJECT_STATUS.md` entry → the one relevant **PROBLEM** entry → the code. It is the same four steps whether you are alone or have help.

A short map of which document answers which question:

You want to know	Read
Am I allowed to do this?	CLAUDE.md
What must never be removed?	CORE_AIM.md
What did I do last?	PROJECT_STATUS.md, top entry
Has this exact bug happened?	V14_FIXES_AND_CODE.md, search by symptom
Shortcuts died again	docs\IF-SHORTCUTS-DIE-AGAIN.md
Which Win32 calls are dangerous	NATIVE_SAFETY.md
How do I ship to the Store	to-publish-in-microsoft-store\START-HERE.md
How does any of this work	this file

Where things stood the day this was written

This will go stale. `PROJECT_STATUS.md`'s top entry is always the authority. But knowing the shape of the open work is what stops you re-solving it, so, as of **2026-09-07**:

- **1.0.106 is on the machine and proven.** `package.json` has moved to 1.0.107 and an installer for it exists in `all-versions\`.
- **PROBLEM 257 is open.** While the dashboard has focus, no keystroke reaches any keyboard hook in this process. Reproduced on 1.0.91, 1.0.100 and 1.0.103, so it is not a regression from one version, and the mechanism is OS-side and undetermined. PROBLEM 259's page-side fallback routes around it; it does not fix it. **Do not treat this as a mystery to be solved from inside** `hook/mod.rs`. Several passes already tried.
- **PROBLEM 262's fix is written and gated but not built or installed.** If your installed version is 1.0.106 or older, the latched-hold outage in 14.6 can still happen to you.
- **PROBLEM 260's forced repair has never executed on hardware.** It is shipped and it is unproven. The first `KEYBOARD DEAF`, `PROVEN` line in a real log will be the first evidence either way.
- **Law 6 is unproven on the most recent builds** — `no hold start ... over own window` followed by `guide_hud: shown over own window` since. That is exactly the state a ship report is supposed to describe as UNPROVEN, and it did.
- **The Microsoft Store submission is waiting on you**, not on code: a Partner Center account, identity verification, the name reservation, and three identity values.
`to-publish-in-microsoft-store\START-HERE.md` is one page and is enough to finish it.
- **The repo has been uncommitted since 1.0.95** (HEAD `ad23e00`). The first commit will be unusually large. That is expected, not a mistake.

14.3 How to use any other AI well

You will use another AI. That is fine and it is sensible. But a general assistant knows nothing about this project, and its instinct on almost every question here is wrong — because the obvious answer for a normal app is the wrong answer for a global keyboard hook.

Treat it as a fast, confident junior who has never seen this codebase. That framing gets it right.

What to paste in, every time

Three things, in this order, before you ask anything:

- **The laws from** `CLAUDE.md` — the keyboard-hook laws, the window rules, the testing laws, the hard rules. Not the whole file if it will not fit; those sections, at minimum, and always all seven hook laws together.
- **The ONE relevant PROBLEM entry** from `V14_FIXES_AND_CODE.md`. One. Not twenty. The entry for the area you are touching. It carries the symptom, the root cause, the exact file and the verification method — which is exactly the context the AI is missing.
- **The actual log lines**, from `%APPDATA%\Spaceadom\debug.log`, from after the build banner. Not your description of them.

Then ask your question.

Here is a preamble you can keep and paste. It is deliberately blunt:

This is a Rust + Tauri v2 Windows app that installs a `WH_KEYBOARD_LL` global keyboard hook. It is NOT a normal desktop app and normal advice will break it. Rules that are not negotiable, and that I will check your answer against: - The hook callback may only touch atomics and one non-blocking channel send. No logging, no locks, no COM, no Tauri, no allocation. If the callback takes longer than `LowLevelHooksTimeout` (1000 ms), Windows stops calling it, leaves the handle valid, and says nothing. - `install_hooks()` installs three hooks in a specific order and that order is the fix for a real bug. Do not reorder it. Do not simplify it. Do not "clean it up". - Injected input is identified ONLY by our own `dwExtraInfo` cookie `0x7A7A7A7A`, never by the `LLKHF_INJECTED` flag. - `GetAsyncKeyState` reports a key we suppress as UP. Never build a failsafe on it for a suppressed key. - I verify claims. Do not tell me something is fixed. Tell me what I should measure, what number proves it, and what number would disprove it. Here are the rules that apply, one prior write-up of this exact area, and the real log lines. Read them before answering.

The three things never to let it do

1. Never let it touch `install_hooks()`. This is the one absolute. Every general-purpose assistant looks at that function and wants to tidy it — three `SetWindowsHookExW` calls in a row, surely the order does not matter. The order is the entire fix for PROBLEM 230. Windows calls the *most recently installed* hook first, and a hook's "am I too slow?" timer covers everything running under it. The witness hook installed last was therefore the slowest-looking hook in the chain, so Windows killed the witness instead of the real hook — and a dead witness can only ever report "all quiet". Measured cost before the fix: **514 false teardowns in one 3.8-hour session** (2026-09-01 baseline, fixed in 1.0.96 on 2026-09-04). Full account: `docs\IF-SHORTCUTS-DIE-AGAIN.md`.

If an AI proposes any edit to that function, paste `IF-SHORTCUTS-DIE-AGAIN.md` at it and ask it to explain the ordering back to you first. If it cannot, it does not get to edit it.

2. Never let it "simplify" the two updater manifests into one. An NSIS install and an MSI install are updated by different installers reading different manifests (`latest.json` and `latest-msi.json`).

Feeding an NSIS install the `.msi` is PROBLEM 129/244. It looks like duplication. It is not.

3. Never let it remove a feature to make something build. That is `CORE_AIM.md`'s prime directive and it is there because it has been tried. If a `CORE_AIM.md` feature is broken, it gets fixed natively. It never gets deleted.

Make it prove things the way this repo proves things

This is the habit that matters most, because a confident wrong answer costs you a day and a hedged right answer costs you nothing.

When an AI says something is fixed, ask for the same three proofs this repo has always demanded:

- **"Which long log sentence from your change should I find in the freshly built exe?"** Then check it, with the ASCII marker scan from section 5 — and check it in the *fresh* exe first, before its absence anywhere else means anything.
- **"What was the rate before and what is the rate now, over what duration?"** For anything intermittent. "It seems better" is not an answer.
- **"What measurement would show you were wrong?"** If there is no such measurement, it has not made a claim — it has made a guess with confidence attached.

And keep the one sentence that generalises all of it: **a check that cannot produce a negative result is not a check.**

What an AI is genuinely good at here

Do not over-correct into using it for nothing. It is very good at:

- explaining a Rust compiler error and what the borrow checker wants;
- writing a pure function plus its unit tests (the kind `cargo test --lib` covers);
- CSS and layout work in `src/`, where the preview harness lets you look at the result in seconds;
- reading a Win32 API's documented behaviour back to you;
- drafting the two documentation entries from your rough notes.

It is bad at: anything about hook ordering, anything about this machine's sandbox, anything intermittent, and any claim that something works.

14.4 When something breaks: five questions, in order

Do not start by reading code. Start here. The order is cheapest-and-most-often- right first, and it has been arrived at the hard way.

Question 1 — Am I even running the build I think I am?

More wasted hours have come from this than from any real bug. A fix that is not installed does not exist.

```
(Get-Item "$env:LOCALAPPDATA\Spaceadom\spaceadom.exe").VersionInfo.FileVersion
```

```
(Get-Item "$env:LOCALAPPDATA\Spaceadom\spaceadom.exe").LastWriteTime
Get-Process spaceadom -ErrorAction SilentlyContinue |
    Select-Object Id, Path, StartTime
```

Three things to notice. The version must be the one you built. The `LastWriteTime` must be after your build. And the **running process's** `Path` must be `%LOCALAPPDATA%\Spaceadom\spaceadom.exe` — if a second copy is running from somewhere else, you have two keyboard hooks fighting over the spacebar and nothing you measure means anything until you kill one.

Question 2 — What does the log say, after the build banner?

```
notepad "$env:APPDATA\Spaceadom\debug.log"
```

Find the last `Spaceadom build - version ...` line. **Everything you assert about "this build" must come from below that line.** Above it is a different program.

Read forward from there for the first `WARN` or `ERROR`. The app logs sentences, not codes, on purpose — most lines explain themselves.

Question 3 — Is it one of the four cheap, common causes?

Four greps, thirty seconds, and they catch most of it:

```
grep safe-mode debug.log
grep "hook liveness split" debug.log | tail
grep "KEYBOARD DEAF, PROVEN" debug.log | tail
grep "WATCHDOG alarm confirmed, but a Space hold is LIVE" debug.log | tail
```

- **Safe mode** — the app came up with no hook and no overlay on purpose, because three launches in a row died young. Looks exactly like a dead hook. See 14.6.
- `hook liveness split` with `mouse:` above zero and `primary_real:0 reference:0` is a named signature: **the keyboard hooks were evicted and the pump is healthy**. Not "nobody typed".
- `KEYBOARD DEAF, PROVEN` means the watchdog already caught it and re-hooked. Read the next `hook liveness split` to see whether the repair took.
- **The latch line** more than once for a single hold is `PROBLEM 262's` stuck modifier. See 14.6.

Question 4 — Has this happened before?

Almost certainly yes. Search by the words that describe what you SAW.

```
grep -n -i "ring did not appear" V14_FIXES_AND_CODE.md | head
grep -n -i "exited 0" V14_FIXES_AND_CODE.md | head
grep -n "^## PROBLEM" V14_FIXES_AND_CODE.md | tail -30
```

Also search `PROJECT_STATUS.md` the same way — it records the *condition* things failed under, which is often the piece that makes a symptom recognisable.

Two hundred and sixty-odd problems are written up. The probability that a new symptom is genuinely new is low, and re-solving a solved problem is the single most expensive thing you can do here.

Question 5 — When did it last work?

If the first four questions found nothing, stop reasoning and start bisecting. `all-versions\` holds every installer ever built.

Back your config up first, install an older `setup.exe`, live with it, halve the interval. Full recipe in section 11, step 3. Then write down **last good and first bad, with timestamps**, in `PROJECT_STATUS.md`. "Last good 2026-09-05 14:11, first bad 15:35" is the single most useful sentence you can write about an intermittent bug — that exact pair is what turned PROBLEM 257 from a mystery into a diff.

14.5 Rolling back to a known-good build, in two minutes

Two ways. Use whichever applies.

The in-app way (only after an automatic update)

Open the dashboard → the gear (bottom-left) → **About** → the button reading "Roll back to 1.0.xxx".

It appears only when there is something to go back to. The updater archives the installer it *replaced* into `%APPDATA%\Spaceadom\rollback\` — so the first automatic update a copy ever takes has no rollback, because the installer that put the app there in the first place was never ours to keep. A missing button is correct behaviour, not a fault. A rollback also holds the daily update check off for 24 hours, so the app cannot turn round and re-install what you just removed.

The manual way (always available)

This is the one to memorise. It works no matter what state the app is in.

```
# 1. Stop the running copy. Nothing else may run before this.
taskkill /IM spaceadom.exe /F

# 2. Back up your bindings, because an older build may not understand a
#    newer config file.
Copy-Item "$env:APPDATA\Spaceadom\config.json" "$env:USERPROFILE\Desktop\config-backup.json"

# 3. Install the known-good version. NSIS setup.exe ONLY - never the .msi.
D:\Claude-Projects\SpaceToggle-V14\all-versions\Spaceadom_1.0.99_x64-setup.exe /S

# 4. Start it.
Start-Process "$env:LOCALAPPDATA\Spaceadom\spaceadom.exe"

# 5. Confirm you got what you asked for.
(Get-Item "$env:LOCALAPPDATA\Spaceadom\spaceadom.exe").VersionInfo.FileVersion
```

Two rules attached to that block, both paid for:

- **Only ever the `setup.exe` files from `all-versions\`.** Do not double-click a `.msi` from that folder. A `.msi` double-clicked on 2026-08-30 registered the per-user install folder as its own, and that registration is what `msiexec /X` later acted on when it deleted the live app.

- **Never trust the exit code.** Step 5 is not optional. `setup.exe /S` has exited 0 having replaced nothing four separate times.

If Spaceadom will not start at all and you need your machine back this second:

```
taskkill /IM spaceadom.exe /F
Remove-ItemProperty -Path "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run" -Name Spaceadom
```

That stops it and stops it coming back at logon. Your config is untouched; nothing is lost. Fix it when you have time.

14.6 The five disasters, and how to recover from each

Each of these has happened exactly once. Each cost a day or nearly did. None of them will be a mystery the second time.

Disaster 1 — An installer that replaced nothing

What you see. The build succeeds. `setup.exe /S` exits 0. The fix is not in the app. You test, conclude the fix does not work, and throw away a fix that was correct.

What happened. Four separate installs did this (PROBLEM 127). Spaceadom autostarts, so it is **always running during its own upgrade**. Windows' Restart Manager needs to ask "may I close these files?" — a question a silent install can never ask — so the file replacement is deferred to a reboot that may never come, and the installer still exits 0.

Why it will not silently recur. The NSIS `installerHooks` `PREINSTALL` macro runs `taskkill /F /T /IM spaceadom.exe` before any file is touched — the `/T` matters, because it takes the `WebView2` child processes with it — then sleeps 1500 ms. `install-real.cmd` kills it again for good measure, and `wix\main.wxs` gained a `util:CloseApplication` element for the MSI leg. Confirmed fixed 2026-08-17 19:11: 1.0.41's `setup.exe /S` run over a running 1.0.40 exited 0 **and** the on-disk binary actually changed, by version stamp and content marker.

How to recognise it. The installed exe's version stamp or `LastWriteTime` does not match your build.

Recovery.

```
taskkill /IM spaceadom.exe /F
Start-Process explorer.exe -ArgumentList 'D:\Claude-Projects\SpaceToggle-V14\scripts\install-real.cmd'
# then read install-check.txt in the repo root
```

The permanent rule. Verify the installed exe by version stamp **and** by an ASCII content marker before believing a fix shipped. A verdict taken on an unverified build is worse than no verdict, because it discredits a fix that worked.

Disaster 2 — A repair that deleted the app

What you see. Spaceadom is gone. Not broken — gone. The install folder is empty. Only `%APPDATA%\Spaceadom` survived.

What happened, on 2026-09-04. The dashboard's rival-install banner offered to remove what looked like a leftover per-machine registry entry. `repair()` ran `msiexec /X{C68DC702-9414-421F-A3E4-12EDBBAD76C5}`. That product's recorded `InstallLocation` was the **live per-user folder**, because a `.msi` double-clicked days earlier had registered those files as its own components. Restart Manager failed to close the running app (`RestartManager 10010`, "SID does not match") and Windows Installer deleted the registered files anyway — the live `spaceadom.exe` — and logged `MsiInstaller 1034 "removed the product ... status 0"`. A successful removal, by its own account.

Recovery. Straightforward, because your settings live somewhere else:

```
D:\Claude-Projects\SpaceToggle-V14\all-versions\Spaceadom_1.0.106_x64-setup.exe /S
Start-Process "$env:LOCALAPPDATA\Spaceadom\spaceadom.exe"
```

Your profiles and bindings come back with it — `config.json` is in Roaming and was never touched.

The permanent rule, now enforced in code. A leftover entry is removed from the **registry only**, never through the Installer. `msiexec` may only ever be aimed at a product whose recorded location is a different directory that is not an ancestor of ours, and even then with `/qn REBOOT=ReallySuppress MSIRESTARTMANAGERCONTROL=Disable`. That decision now lives in one pure, unit-tested function, `rival_install::plan_removal` (PROBLEM 244), specifically so it can be tested without running it.

Generalise it: *a removal that trusts a registry entry's own description of what it owns is a removal aimed by the thing being removed.*

Disaster 3 — A stuck modifier latch

What you see. No ring at all. Not "the ring died mid-press" and not "a ring got stuck on screen" — **no ring from either path, until you restart the app**. Space still types spaces. Nothing crashes.

What happened, on 2026-09-07. You held Space over Spotify, tapped Space+RightAlt to cycle a profile, and the keyboard hook was evicted on the very next callback. The Space-UP that would have cleared `MODIFIER_ACTIVE` therefore never arrived, and the flag stayed latched. That one latch then independently blocked three separate recovery paths: the reaper could not clear the stale hold, the watchdog's proven-deaf forced repair returns `None` while a hold is latched, and guard 2 of the own-window fallback refuses to raise a ring while it is set — so the page-side backup path was wedged shut too. **The ring was gone from both witnesses for 166 seconds**, and it came back only by luck when a stray callback hit a 30-second bound. Written up as PROBLEM 262 and fixed in code the same day, after 1.0.106 — `hook/mod.rs` gained a deafness-aware reap reason, a bounded deferral episode, teardown-before-repair, a 30-second belt-and-braces unlatch and a fallback-vs-fallback guard.

Check whether the build you are running actually has that fix, because the entry was written with the gates green but explicitly NOT BUILT and NOT INSTALLED. If your installed version is 1.0.106 or older, it does not.

Two sentences from that entry worth keeping: *an instrument that can only be read by the thing that has failed is not an instrument*, and *a bound whose start stamp is cleared by a condition unrelated*

to the thing it bounds is not a bound.

How to recognise it. In this order:

```
grep "WATCHDOG alarm confirmed, but a Space hold is LIVE" debug.log
grep "stale-hold-reaped-because-the-keyboard-is-proven-deaf-spaceadom" debug.log
grep "modifier-active-latched-past-the-bound" debug.log
grep "repair-tore-down-a-hold-that-predated-it-spaceadom" debug.log
```

Holds `protected this session`: climbing 1, 2, 3, 4 for a **single** hold is the fingerprint. The three markers below it are the three bounds that now break the latch. If none of them appears and the ring is still gone, something new is holding the latch — and that is a new PROBLEM entry, not a re-run of this one.

Recovery right now. Restart the app. That is genuinely all:

```
taskkill /IM spaceadom.exe /F
Start-Process "$env:LOCALAPPDATA\Spaceadom\spaceadom.exe"
```

Disaster 4 — A hook gone deaf

What you see. Shortcuts stop working. Sometimes everywhere; sometimes only while the Spaceadom dashboard itself has focus. It comes back on its own after a while, or after a restart. Nothing crashes and nothing appears in the UI.

This one has three different causes and they are genuinely different bugs. Read `docs\IF-SHORTCUTS-DIE-AGAIN.md` before doing anything else — it exists precisely so this is not re-diagnosed from scratch, and it lists three plausible-sounding explanations that were tested and disproved, with the numbers, so nobody has to re-run those tests.

The mechanism underneath all three is one documented Windows behaviour worth knowing by heart: **a `WH_KEYBOARD_LL` callback that overruns `LowLevelHooksTimeout` stops being called and keeps a valid handle.** No message. No error. No return code. `UnhookWindowsHookEx` on it still succeeds. From inside the process, an evicted hook and a hook nobody has typed into are the same observation. And `WH_MOUSE_LL` is a **separate hook with its own timeout record** — it keeps firing at 30–60 Hz while the keyboard hooks are dead, which makes the app look healthy from every clock except the keyboard's own.

Tell the three apart like this:

Signature in the log	Which one
<code>genuinely fired (N total) frozen while the app is in use</code>	PROBLEM 230 — the witness hook is being killed; <code>install_hooks()</code> has been reordered
<code>mouse: above 0 with <code>primary_real:0</code> <code>reference:0</code>, and only while our own window has focus</code>	PROBLEM 257 — keys never reach any hook in the process. Still open
<code>mouse: above 0 with <code>primary_real:0</code> <code>reference:0</code>, any window, and the watchdog printed nothing for minutes</code>	PROBLEM 260 — the watchdog was waiting for the mouse to fall silent

Be clear-eyed about the middle row: PROBLEM 257 is not fixed. While the dashboard has focus, neither keyboard hook is called at all — measured on 2026-09-06 at 00:54:52 as `mouse:2705 primary_real:0 reference:0`, our window foreground in 60 of 60 samples, with every suppression-gate counter reading 0. Re-hooking does not cure it, and the owner reproduced it on 1.0.91, 1.0.100 and 1.0.103, so it is not a regression from any one version. The mechanism is OS-side and undetermined. What exists is a **route around it**: PROBLEM 259's own-window fallback runs the same tap/hold/combo state machine in the dashboard page, and PROBLEM 261 (fixed in 1.0.106) taught cursor tracking and click-to-launch to recognise a fallback hold as a hold. So the app works inside its own window; the hook still does not. Do not spend a week trying to "fix" this from inside `hook/`. Several already have.

If the ring appears inside the dashboard but **the cursor does not track and clicking a chip launches nothing**, that is PROBLEM 261's shape, and 1.0.106 or newer has the fix:

```
grep "hud-pointer: ARMED chip" debug.log
grep "engine: pointer activation" debug.log
```

Recovery. Least destructive first:

- **Click another window.** For PROBLEM 257 the keys come back the instant something else takes focus.
- **Settings → the hook status control → reinstall the hook** (the `reinstall_hook` command). A repair is an unhook plus a fresh `SetWindowsHookExW`; the log prints old → new `HHOOK` values to prove it. **A handle that did not change means the install failed** — never argue about whether a repair happened again, read the handles.
- **Restart the app.**

Then check the repair took, because a repair that changes nothing is the interesting case:

```
grep "hook liveness split" debug.log | tail
```

`primary_real` above 0 in the split *after* a forced repair means the reinstall cured it. Still 0 means the drop is upstream of every hook in this process, and re-hooking is not the answer — read the backoff line.

Two readings of that log that are wrong and will catch you. `watchdog- reinstalls:1` twice in a row does **not** mean the counter is stuck: that field is drained every 60 seconds, so two windows reporting 1 means one reinstall in each. And `repair #1 this session` printed three times was a naming bug, not a missing repair — a number printed beside the word "session" must read a counter nothing drains.

A caution that is easy to forget. Since PROBLEM 259 the dashboard page has its own fallback path that feeds Space to the engine directly, so **a ring you saw inside the dashboard is no longer evidence that the hook is alive**. Read which line produced it:

```
grep "hold start (hold #" debug.log | tail      # the HOOK saw the hold
grep "own-window fallback:" debug.log | tail    # the PAGE saw it instead
```

Disaster 5 — A config that looks stale

What you see. You open `config.json`, and it is months old. Bindings you know you changed are not in it. Meanwhile `debug.log` right beside it is tracking the clock to the second.

What happened. You read it from a shell running inside an MSIX container. The container's view of the filesystem is a **copy-on-write union**, not a redirect: a file the container has ever *written* exists in its private store and shadows the real one; a file it has never written is not there and reads straight through to the real file. So `config.json` came back 47,754 bytes dated 18 August against a real file of 62,463 bytes, while `debug.log` and `picker-cache.json` in the same folder were live. Observed twice, 2026-08-26 and 2026-08-27; explained 2026-09-05.

This is the trap that makes this one dangerous: the folder cannot look stale as a whole. The usual tell is absent. Per-file cross-checking is the only safe habit.

How to read the real one:

```
Start-Process explorer.exe -ArgumentList 'D:\Claude-Projects\SpaceToggle-V14\scripts\config-check.cmd'  
# then read config-check.txt and _config-live-copy.json in the repo root
```

That script is read-only by design — it copies the config out to `D:\`, a drive the container does not redirect, and never writes back into `%APPDATA%`.

Cross-check the byte size against what `debug.log` says was last saved. If they disagree, you are reading a shadow.

And the rule that generalises it, which is the most important sentence in this whole guide: printing `%LOCALAPPDATA%` proves nothing, because MSIX redirects at the filesystem layer and the path *string* is identical inside and out. An agent did exactly that on 2026-08-27, concluded it had escaped, and was right only by luck. **The only valid proof is a differential** — read the same path string from both contexts and compare what comes back.

A last note on this one, and it matters more than it sounds. Unexpected values in a live config usually mean **the owner has been using the app**, not that something broke it. Never write to the live `config.json` to test something. Copy it out, work on the copy.

14.7 Keeping the documentation habit alive, alone

This is the part that decays first when nobody is watching, and it is the part that everything else in this guide rests on.

The rule has not changed: **every solved problem gets two entries, and neither is optional.** One in `PROJECT_STATUS.md` (dated, newest at top, append-only, what happened and under what condition it failed), one in `V14_FIXES_AND_CODE.md` (symptom → root cause → exact file → the actual code → how it was verified, plus a "generalise this" line when the bug has a class).

Your own reason for the rule, in your words: *without documentation an AI has to start from scratch, and that costs a huge number of tokens.* That reason is about to get stronger, not weaker. When you are paying a general assistant to read one PROBLEM entry instead of your whole codebase,

the entry is the difference between a useful answer and an expensive guess.

But here is the sharper reason, now that you are alone: **you are the AI now**. In six months you will come back to this project with exactly the same problem a fresh session has — no memory, no context, a symptom and 40,000 lines of code. The entry you write today is written to yourself.

How to keep it up when there is no one to enforce it.

- **Write it before the build, not after.** Section 12 puts the two entries at step 6, before build-and-install, and that ordering is deliberate. After the build you are tired, it works, and you want to stop. Before the build you still remember the condition it failed under — which is the one detail that makes an entry worth having.
- **Write badly rather than not at all.** A rough, honest, dated entry beats a polished one that never gets written. You can tidy it later; you cannot reconstruct the condition later.
- **The test for a finished entry is one question:** *could another person, or another AI, apply this fix from this file alone, without opening the codebase to search for it?* If not, it is not finished.
- **Record the class, not just the instance.** "An ID rule that sets `display` needs its own `#id[hidden]` companion" is reusable forever. "The profile popover was open" is one night.
- **Write down the wrong diagnoses too.** The "measurement traps" sections exist because two false bug reports were nearly filed, and they have saved more time than several of the fixes. A note saying "we thought it was X, here are the numbers that disprove X" stops the next session re-running that test.
- **Never delete an entry.** Append-only, always. And when you write to these files, **a prepend is an overwrite** — write the new head to a temp file, append the old file to it, then move it into place. On 2026-09-07 a one-liner that read a file and wrote it back in the same statement had its read fail and its write succeed, and `PROJECT_STATUS.md` went from 817,401 bytes to 6,121.
- `git push` **after every session that wrote to them.** Git is the only thing protecting these files, and the tree has sat uncommitted for long stretches before.

14.8 What to learn next

Three things. In this order. Each one has a specific reason for *this* app — this is not a general "learn programming" list.

1. Rust ownership, borrowing and `Arc/Mutex/RwLock`

Why it matters here. Almost every Rust error you will hit in this codebase is an ownership error, and almost every one of them is the compiler stopping you from doing something that would actually have been a bug. The config is `Arc<RwLock<AppConfig>>` and shared across threads; the hook thread and the engine communicate by moving small values down a channel *specifically* because they must not share memory. Once ownership clicks, the architecture of this app stops looking like ceremony and starts looking like the only sane arrangement.

Where. *The Rust Programming Language* ("the book", free at doc.rust-lang.org/book), chapters 4, 15 and 16. Three chapters, not the whole book. Read them with `hook/mod.rs` and `lib.rs` open

beside you.

2. Win32 message loops, and low-level hooks in particular

Why it matters here. The single most surprising line in the codebase is the `while GetMessageW(...)` loop in `hook_thread_main` that appears to do nothing. It is what makes the hooks fire at all — Windows only calls a low-level hook callback on a thread that is pumping messages. Delete it and the app goes silent with no error. Every one of the seven hook laws is a consequence of how that mechanism works: the timeout, the chain order, the eviction that leaves a valid handle, the injected-input flag being untrustworthy.

Where. Microsoft Learn's own pages: "About Messages and Message Queues", `SetWindowsHookExW`, `LowLevelKeyboardProc`, `CallNextHookEx`. Read the "Remarks" sections — that is where the behaviour that costs you days is written down in one sentence. The skill file `.claude\skills\arpons-windows-apps-building-skills\references\win32-keyboard-hook.md` in this repo is the condensed version.

3. The Tauri v2 documentation

Why it matters here. Every UI feature you add crosses the boundary the same way: a `#[tauri::command]` in Rust, registered in `generate_handler!`, called from TypeScript with `invoke()`; and events going the other way with a global `emit` and a `listen()`. The two things that bite are both documented and both have bitten already — a window that is not listed in `src-tauri\capabilities\default.json` is **deaf**, every `listen()` in it rejecting silently; and a command missing from `generate_handler!` is invisible, with `invoke` simply rejecting. Knowing the model means you recognise both in seconds instead of hours.

Where. `v2.tauri.app` — the Calling Rust, Calling the Frontend, and Capabilities/Permissions pages. That is most of what you need.

A fourth, if you want it

Reading Windows Event Viewer and your own log properly. Not glamorous, but this project's two worst days were both misread evidence rather than hard bugs — a build-time WiX validation event read as an install, and a drained counter read as a repair that never happened. Being fluent in your own instruments is worth more here than another language.

14.9 The short version

If you remember nothing else from this section:

- **Read the log first.** `%APPDATA%\Spaceadom\debug.log`, below the build banner.
- **Check you are running what you think you are running.** Version stamp, `LastWriteTime`, and the running process's path.
- **Search `V14_FIXES_AND_CODE.md` by symptom before touching code.** It has almost certainly happened before.
- **Never let anything reorder `install_hooks()`.**

- **Never trust an installer's exit code.**
 - **A check that cannot produce a negative result is not a check.**
 - **Two documentation entries, every time, before the build.**
 - `all-versions\` and the pen drive mean no day is unrecoverable.
-

15. Trap glossary

Short entries, alphabetically useless but ordered by how likely they are to catch you. Each one is a thing that looked like a bug and was not, or looked like health and was not. All of them have happened here.

The MSIX container shadow

What it looks like. A file in `%APPDATA%\Spaceadom` or `%LOCALAPPDATA%` is months out of date, while a file beside it in the same folder is current to the second. Or: an install "succeeds", every verification passes, and the real machine has no app at all.

What it is. Some shells — the agent shell on this machine especially — run inside an MSIX container whose filesystem view is a copy-on-write **union**. A file the container has ever written exists in its private store and shadows the real one. A file it has never written reads straight through to the real file. `HKCU` is virtualised the same way.

Why it is nasty. The folder cannot look stale as a whole, so the usual tell is absent. And a verification performed by the sandboxed process cannot detect the sandbox — every check agrees with itself and is wrong.

What it cost. An entire session's worth of installs. Every one of them looked perfect from inside the shell; the real machine had no app and no Run key. The way it surfaced was the owner saying *"I restarted my laptop and it didn't come up."* That is PROBLEM 143, and it is why `install-real.cmd`, `config-check.cmd` and the two probe scripts all exist and all write their findings to `D:\`, a drive the container does not redirect.

The escape. Run through `explorer.exe`, which is outside the container:

```
Start-Process explorer.exe -ArgumentList 'D:\Claude-Projects\SpaceToggle-V14\scripts\install-real.cmd'
```

The test that does NOT work. Printing `%LOCALAPPDATA%`. MSIX redirects at the filesystem layer, so the path *string* is byte-identical inside and out. The only valid proof is a **differential**: read the same path from both contexts and compare. The measurement that settled it (2026-08-27): the same path returned v1.0.53 / 14,109,184 bytes in-sandbox and v1.0.86 / 18,868,224 bytes via `explorer.exe`.

One more, in case you reason about MSIX later. `GetCurrentPackageFullName` in that same shell returns `APPMODEL_ERROR_NO_PACKAGE` — a process can be inside the redirection view with no package identity of its own. So `packaged::is_packaged()` is not a test for redirection and must never be used as one.

The marker trap

What it looks like. You scan the freshly built exe for a string from your change. It is not there. You conclude the fix did not build.

What it is. Not every Rust literal survives into the binary contiguously. A short literal that is only ever copied into a `String` may be assembled at runtime by overlapping immediate stores and never sit together on disk. `st-hud-pointer` — 14 bytes, a thread name — tested **False in a freshly built exe that certainly contained it** (measured 2026-08-27); at offset `0x2C8B3` the compiler materialises it with two overlapping `mov` instructions. Longer names in the same file survived fine in `.rodata`.

Separately, frontend markers do not work at all. Tauri v2 compresses the embedded `dist2` assets, so CSS class names and JS strings are not findable in the exe. `st-hud-glow` tests False in a binary that certainly contains it (measured 2026-08-20).

The rules. Use a long `log::format string`, never a short identifier. And **confirm the marker is present in the fresh build BEFORE its absence anywhere else means anything** — otherwise a `False` reads as "the fix did not ship" when it means "that marker was never findable", and a working build gets thrown away. Keep about twenty old control markers in the scan so a wall of `False` tells you the scan is broken rather than the build. For frontend changes, use the chain instead: `grep dist2\assets\*`, then check the exe's `LastWriteTime` is later than the newest file in `dist2`, then the version stamp.

The self-matching grep

What it looks like. A counter that rises when the app is healthy. Or a grep that finds hits that are not the thing you are counting.

What it is. A log line that tells the reader what to grep for becomes a hit for that grep. The engine's own hold-start line ends with the advice `... grep 'KEYBOARD DEAF, PROVEN'`. — so a counter matching the bare phrase counted eighteen *successful holds* as eighteen deaf events. The counter rose with health. Fixed by matching `hook: KEYBOARD DEAF, PROVEN`, the WARN's own prefix, which the advice text cannot contain.

The same shape caught the fallback line: PROBLEM 259's own-window line quotes the advice `hold start (hold #N) ... over own window`, so a bare `hold start` matches it. A real hook hold reads `hold start (hold #` followed by a **digit**; the quoted advice has the letter **N**. That was the difference between 2 and 0.

The rule. Anchor patterns on something the advice text cannot contain — a module prefix, a digit, a log level.

The drained counter

What it looks like. `repair #1 this session` printed three separate times, so you conclude the repair never actually happened. Or `watchdog-reinstalls:1` in two consecutive windows, so you conclude the counter is stuck.

What it is. `hook diagnostics` swaps its counters to zero every 60 seconds. Those fields are **per-window counts**, not session totals. Two windows reporting 1 means one reinstall in each.

The rule. A number printed beside the word "session" must read a counter nothing drains. The forced-repair count now comes from `FORCED_REPAIRS_TOTAL`, and the line prints **old** → **new** `HH00K values for all three hooks` — a handle that did not change means the install failed. Read the handles; never argue about whether a repair happened.

Bash mangling a value that begins with a slash

What it looks like. `Wrong password for that key`, from a password that is completely correct.

What it is. `MSYS2` — which the Bash tool runs on — rewrites a POSIX-looking value into a Windows path before it reaches a native `.exe`. The updater signing password is base64, so about one in three of them starts with `/`. The rewrite happens as an **argument** (`-p`) *and* as an exported environment variable, quoted or not, with `MSYS_NO_PATHCONV=1` set or not.

The cost. A full day, on 2026-09-05. And it nearly caused something unrecoverable: regenerating the signing key looked like the obvious next step, and that would have stranded every installed copy forever.

The rule. **Any secret handed to a native Windows binary goes through PowerShell.** Never the Bash tool.

```
$env:TAURI_SIGNING_PRIVATE_KEY = (Get-Content "src-tauri\tauri\spaceadom.key" -Raw)
$env:TAURI_SIGNING_PRIVATE_KEY_PASSWORD = (Get-Content "src-tauri\tauri\spaceadom.key.password.txt" -Raw).Trim()
```

The CP1252 curly-quote parse failure

What it looks like. A PowerShell script produces no output at all. In this case, one line: `PROOF STEP PRODUCED NOTHING - powershell never ran`. It reads like a plumbing hiccup.

What it is. `install-proof.ps1` is saved as UTF-8 **with no BOM**, and `install-real.cmd` invokes it with `powershell` — Windows PowerShell 5.1, not `pwsh`. Windows PowerShell reads a BOM-less file as **CP1252**. A UTF-8 em dash (U+2014) is the three bytes `E2 80 94`; CP1252 decodes that third byte `0x94` as U+201D, a right double quotation mark — and **PowerShell accepts a curly quote as a string delimiter**. So a double-quoted string containing an em dash terminates *at the em dash*, and everything after it is parsed as code:

```
+ $ownWindow = "FAIL â€" no 'Spaceadom build ... version $ver' bann ...
+           ~~
Unexpected token 'no' in expression or statement.
```

A parse error kills the whole file before its first statement, so nothing ran and no output file was created.

Note the asymmetry that hid it. The same em dash inside a **single**-quoted string is harmless, because U+201D does not close a single-quoted string. Two of the five affected lines were single-quoted and looked identical to a reader.

What it cost. The law-6 own-window proof block had been written the previous night alongside the law itself, and **had never once executed**. The release it was written to gate would have shipped with the gate absent.

The rules. Keep PowerShell scripts **pure ASCII** — the same rule the marker lists already follow for the exe scan, applied to the script itself. And: **a proof step whose only failure signal is one line of .cmd output reads as a plumbing hiccup, not as "the proof does not exist."** Make proof failures loud.

WiX validation events that look like installs

What it looks like. You build, then check the Windows Application event log, and find `MsiInstaller 1033` — "installed the product". You conclude a ship step installed the .msi behind your back.

What it is. WiX's `light.exe` validates the package it has just written by running it through the Windows Installer engine. That logs `11707 + 1033` five to ten seconds after the .msi file's own mtime, **on every single build**. Measured 2026-09-04 across 45 versions.

The tell that separates them. A genuine install or uninstall is bracketed by a **transaction pair**: `1040` "Beginning a Windows Installer transaction" and `1042` "Ending...", naming either the .msi path or the ProductCode. The build-time `1033s` have neither.

For the record, the one real per-machine MSI install on this machine was 2026-08-30 15:09 — `1040/1042` naming `Spaceadom_1.0.94_x64_en-US.msi` inside WhatsApp Desktop's transfers folder, because a shared build got double-clicked. That install created the registration `msiexec /X` later acted on when it deleted the live app.

A read-modify-write one-liner truncating an append-only log

What it looks like. `PROJECT_STATUS.md` goes from 817,401 bytes to 6,121.

What it is. On 2026-09-07 a one-liner read the file and wrote it back **in the same statement** — the pattern that looks like "prepend a new entry". Its read failed. Its write succeeded. The file was rebuilt from a week-old git copy plus session transcripts, and the recovery notice at the top of the file is honest about what may still be missing.

The rule. A prepend is an overwrite. The safe form is always three steps:

```
# write the new head to a temp file
Set-Content -Path _new-head.md -Value $newEntry -Encoding UTF8
# append the OLD file to it
Get-Content PROJECT_STATUS.md -Raw | Add-Content -Path _new-head.md
# only then move it into place
Move-Item _new-head.md PROJECT_STATUS.md -Force
```

And check the result is **larger** than what you started with before you believe it worked.

The neighbouring rule: never delete, rename or overwrite a file you did not create — stale-looking or not, `.tmp/.bak/.old` or not. A `PROJECT_STATUS.md.tmp` was deleted on 2026-09-04 on

exactly that reasoning; the file it looked like a leftover of was 580 KB of append-only log, and only luck decided whether that `.tmp` was garbage or the only copy of an in-flight write. The rule has no judgement clause on purpose — "it looked stale" is the sentence that precedes every one of these.

An intermittent bug proved fixed by one good hour

What it looks like. You ship a fix, use the app for an hour, nothing goes wrong, and you write "fixed".

What it is. If a fault appeared roughly twice an hour, an hour without it is barely evidence. It is what you would expect a quarter of the time from doing nothing at all.

The rule. State the rate before, the rate after, and the duration. `21.0 deaf-minutes per 100 active minutes` is a claim. "It seems better" is not. The real numbers this project runs on look like that: 514 false teardowns in 3.8 hours as a broken baseline; 16 alarms in 38 minutes as the next baseline; near zero as the target.

The companion rule, which costs nothing and is worth more than any fix: never report something as fixed, working or verified unless you observed it working. Label untested things **UNTESTED**, in capitals if it is going in a ship report. That single habit is what keeps this project trustworthy to yourself six months from now.

And one that is not a trap but reads like one: safe mode

What it looks like. The app starts, the dashboard opens, and Space does nothing. No ring. No toasts. `debug.log` has no `hook: rollover window` line. It looks exactly like a completely dead hook.

What it is. Deliberate. Three launches in a row that started and never stayed alive 30 seconds put the next one into safe mode: no `WH_KEYBOARD_LL`, no overlay window, no display watcher, and a banner. A clean shutdown never counts toward it.

Check for it before diagnosing anything:

```
grep safe-mode debug.log
```

The marker is `safe-mode-entered-after-three-consecutive-startup-crashes-spaceadom`.

To clear it: press "Turn back on" in the banner — it installs the hook immediately, no restart needed — or delete `%APPDATA%\Spaceadom\boot-attempts.json`, or set its `failed_starts` to 0.

End of the developer guide. If you got here by reading straight through: the one thing worth carrying out of it is that almost nothing in this project was ever fixed by being clever. It was fixed by measuring the right thing, writing down what was measured, and refusing to call something done before it was proven. Keep doing that and the rest follows.